

基于互补替代关系的补货定价策略

摘要

针对问题一，本文首先进行数据预处理，将附件 1 和附件 3 的数据汇总到附件 2，随后对附件 2 中的无效数据采取剔除处理，并对类别进行量化处理。为了描述蔬菜六大品类销售量的分布规律，本文引入均值、最大值、最小值、标准差、变异系数、偏度、峰度描述六大品类的统计规律，随后对销售量可视化，随后通过灰色关联分析和 Spearman 相关性分析方法，结果表明：得到除了茄类以外的品类均符合季节性规律和工作日-休息日规律，且相互之间相关性较大，茄类的与各个品类的相关性较低。针对单品的分析，本文绘制时间序列图得到单品的节假日激增规律，并对所有单品同样使用 Spearman 相关性分析，得到单品之间的互补和替代关系，结果详见表 5。

针对问题二，第一小问要求分析销量与定价的关系，本文进行机理分析，建立成本加成定价模型，用历史售价和批发价进行参数估计得到微商的固定成本和每个品类的单位变动成本。建立价格-需求函数，采用不同的非线性回归模型，选择最优的函数作为拟合结果。结果得到：销售总量与售价具有指数相关性，且为**负相关**。第二小问要求制定补货定价策略，本文首先分析菜品补货后的路径，随后以最大化收益为目标，以六大品类的补货量为决策变量，结合损耗率的**随机变量假设**，考虑供需约束，建立非线性规划的优化模型。此外，本文基于历史数据使用时间序列模型预测相关变量作为模型的条件，并以预测销量的置信区间作为决策变量的约束。通过遗传算法求解，结果详见表 6。最后进行模型的检验，得到模型的性能优秀，普适性较强。

针对问题三，题目新增了单品总数和最小陈列量约束，经过筛选得到本文需要考虑 49 种单品，本文引入 0-1 决策变量表示是否引入该单品，以尽量满足市场需求为松弛约束，并在目标函数中加入**惩罚项**，并考虑单品之间的**互补替代关系**，建立**非线性混合整数规划的优化模型**。相比于问题二，问题三样本规模小、预测趋势短，因此本文基于**灰色预测**模型预测批发价格和加成率作为模型的条件。最后求解结果见表 8，并进行模型的检验，验证了模型的合理性。

针对问题四，题目要求我们采集相关数据，以便于商超蔬菜商品的补货和定价决策。本文从竞争冲击、供应链费用、消费者反馈、政策干预四个维度展开，帮助商超实现利润最大化的目标。针对竞争冲击效应，我们考虑了线上销售渠道对线下商超决策的影响。通过在线上平台采集数据计算线上溢价率，我们对上问的补货和定价模型进行了优化，并建议商超在决策时需要考虑到线上菜品的价格变动。

关键字： 非线性规划 灰色预测 混合整数规划 相关性分析 互补替代关系

一、 问题重述

1.1 问题背景

在生鲜超市经营蔬菜类商品时，面临一系列特殊挑战。这些商品通常具有短暂的保鲜期，品质会随着时间的推移而下降。大部分菜品无法隔日出售。因此，生鲜超市必须每天根据商品的历史销售情况和市场需求进行及时的补货。在这种情况下，超市需要深入了解不同商品的销售模式，利用数据分析和市场趋势来指导其补货策略，确保及时提供顾客所需的商品，并优化价格策略以最大程度地减少损失。

1.2 问题重述

现已知某商超近三年的销售数据，其中附件 1 为 6 个蔬菜品类的商品信息，附件 2 和附件 3 记录了该商超在 2020 年 7 月 1 日到 2023 年 6 月 30 日的各商品的销售流水明细与批发价格的相关数据，附件 4 记录了各商品近期的损耗率。要求根据附件相关信息建立相关数学模型，解决以下四个问题：

针对问题一：分析蔬菜类商品不同品类以及不同单品之间的关联关系；根据附件数据，分析并描述蔬菜各品类及单品销售量的分布规律和相互关系。

针对问题二：根据附件提供的历史数据，要求最大化商超收益，以品类为单位，制定各蔬菜品类在未来一周的补货计划和定价策略。

针对问题三：考虑销售空间的有限性，依据 2023 年 6 月 24-30 日的可售品种，进一步制定单品的补货计划，给出 7 月 1 日的单品补货量和定价策略，要求尽可能满足市场需求的条件下最大化商超收益，并要求单品总数在 27-33 个范围内，且各单品订购量要大于等于陈列量 2.5 千克。

针对问题四：为了更好地制定补货和定价策略，请说明还需要综合哪些其他因素，并给出意见和理由。

二、 问题分析

2.1 问题一的分析

针对问题一，本题要求我们分析蔬菜类商品不同品类以及不同单品之间的关联关系以及分布规律，首先要进行数据预处理，将附件 1 和附件 3 的数据链接到附件 2，随后对附件 2 中的无效数据进行剔除，并对类别进行量化处理。为了描述蔬菜六大品类销售量的分布规律，本文引入均值、最大值、最小值、标准差、变异系数、偏度、峰度描述

六大品类的统计规律，随后分析销售量的时序规律，绘制近三年的销售量与时间的示意图，并通过灰色关联分析和 spearman 相关性分析方法，得出关联关系。针对单品的相互关系，我们考虑求解所有单品之间的相关系数，以此分析类间、类内的互补替代关系。

2.2 问题二的分析

针对问题二，第一小问要探究品类销售总量与售价之间的关系。首先进行机理分析，根据历史销量数据对成本加成定价模型进行参数估计，求出商超的固定成本和每一品类的单位可变成本。绘制售价与销量的散点图，发现大致呈负相关关系。随后通过非线性回归进行拟合，得到价格-需求的函数表达式。第二小问要求制定定价补货策略的优化模型，以微商利润最大化为目标，建立基于非线性规划的优化模型，并进行模型的检验验证合理性。

2.3 问题三的分析

问题三与问题二是递进的关系，新增了有限销售空间和最小陈列量的约束，需要 we 们根据 2023 年 6 月 24-30 日的数据，进一步制定商超在 2023 年 7 月 1 日单品的定价与补货策略。需要注意的是本题使用的数据时间跨度比较小，属于小样本数据。与问题二不同的是，问题三的单品数有 49 个，且问题三要求尽量满足市场对各蔬菜单品的需求。为了方便表示销售单品数 $[27, 33]$ 的限制，本文引入 0-1 决策变量表示是否对该单品补货。同时，为了尽量满足市场需求，我们考虑在目标函数中引入惩罚项，以增加模型的灵活性。

2.4 问题四的分析

针对问题四，题目要求我们采集更多相关数据，帮助商超制定蔬菜商品的补货和定价策略，以实现收益最大化的目标。影响补货和定价决策的因素除了固定成本、变动成本、损耗率等因素之外，还可以从竞争冲击、供应链费用、消费者反馈、政策干预等多方面进行挖掘。

三、模型假设

本文提出以下合理假设：

1. 假设蔬菜类商品的毛利率在小于等于 85% 范围内属于正常数据。
2. 不考虑商超由于退货引起的成本消耗。
3. 假设隔天销售的品类不易变质且需求稳定，不存在打折销售和积压浪费的情况。
4. 假设企业从不同供应来源补货的单品由于产地、品质不同等原因分别销售。

四、符号说明

符号	说明
s_i	第 i 个品类的销售价格
x_i	第 i 个品类的补货量
α_i	第 i 个品类的加成率
b_i	第 i 个品类的批发价格
C_{i1}	第 i 个品类的总的固定成本
C_{i2}	第 i 个品类的单位变动成本
q_i	第 i 个品类的实际销售量
k_i	第 i 个品类的损耗率
d_i	第 i 个品类的折扣率
p_i	第 i 个品类的折扣比例

五、问题一的分析与解答

5.1 数据预处理

5.1.1 相关表单的汇总与量化处理

由于题目所给的附件数据比较分散，为方便分析，我们使用 Python 将附件 1 的分类编码与分类名称和附件 3 的单日批发价格链接汇总至附件 2。为了更好地进行关联性分析，我们对附件 2 中的“单品名称”，“分类名称”，“是否打折”等指标进行量化处理，赋值 0, 1, 2 等等。

5.1.2 异常值的处理

(1) 关于毛利率的异常值的处理

查看附件 2 数据，我们注意到：有部分单品的批发价过小，但是以正常价格进行销售；部分单品的销售单价过大，但是以正常价格补货。部分异常值如下所示：

表 1 部分异常数据展示

销售日期	销量	销售单价	单品名称	分类名称	批发价格	利润率
2022-08-25	0.502	9.2	上海青	花叶类	0.01	99.89%
2022-12-13	0.238	16	高瓜 (2)	水生根茎类	0.01	99.94%
2022-10-17	0.298	19.8	红灯笼椒 (2)	辣椒类	0.86	95.66%
2020-09-05	0.042	12	小米椒	辣椒类	0.59	95.08%

结合生活经验，我们假设蔬菜类商品的毛利率在小于等于 85% 范围内属于正常数据，定义毛利率公式如下所示：

$$GM = \frac{R - C}{R} \quad (1)$$

其中， GM 表示该单品的毛利率， R 表示销售单价， C 表示批发价格。定义 $GM > 85\%$ 时，毛利率过大，视为无效数据，本文作剔除处理。

(2) 关于打折情况的异常值的处理

针对打折，我们发现有部分数据在同一天中打折的售价大于等于不打折的售价，有部分数据在不打折的情况下售价小于批发价的，视为无效数据，本文作剔除处理。

5.2 针对六大品类的分析

5.2.1 销售量的分布规律

(1) 品类销售量分布统计

根据附件 1，绘制六大品类销售量统计如下：花叶类蔬菜销售量最大，为日常所需菜品种类，其他品类销售量差异性不明显。

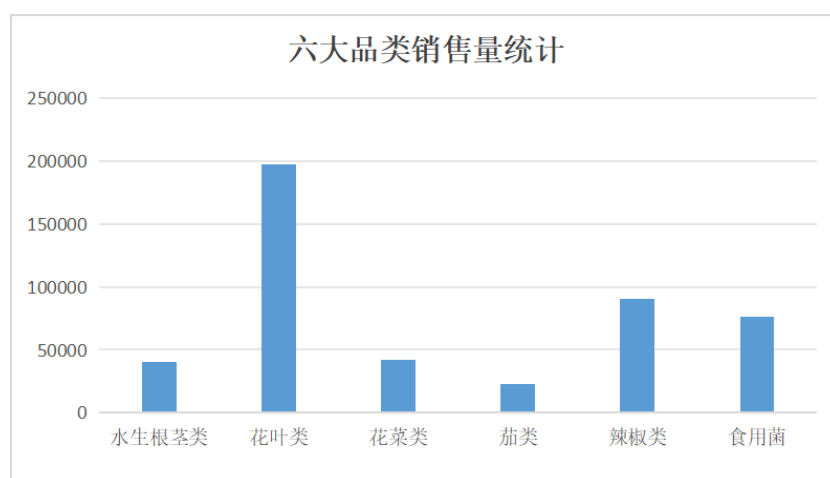


图 1 六大品类销售量统计

从直方图中可以看出，在这些品类中，花叶类和食用菌是最主要的品类，品种数量分别为 100 和 72，说明这两种品类在市场上具有很高的多样性。

(2) 描述统计量

对各个品类内的每日销量进行分类汇总，利用 Minitab 软件求解针对六大品类的销量的描述性统计量，结果如下表所示：

表 2 六大品类的描述性统计分析结果

变量名	最大值	最小值	平均值	标准差	峰度	偏度	变异系数
水生根茎类	296.792	0	36.98	31.395	12.068	2.492	0.849
花叶类	1264.738	0	180.27	86.543	24.477	2.652	0.48
花菜类	186.155	0	38.029	22.862	4.055	1.468	0.601
茄类	118.931	0	20.418	13.569	5.202	1.563	0.665
辣椒类	604.231	0	82.586	53.679	18.178	3.092	0.65
食用菌	511.136	0	69.376	48.727	16.515	2.939	0.702

从表中数据可以分析得到：

- **品类之间的差异：**不同品类之间存在显著的销售量差异。花叶类的平均销售量 180.27 明显高于其他品类，而茄类的平均销售量 20.418 相对较低。
- **品类内部的离散度：**辣椒类和花叶类的标准差较高，分别为 86.543 和 53.679，表明它们的销售量数据相对分散；而水生根茎类的标准差也较高，为 36.98，说明其销售量波动较大。
- **偏度和峰度：**辣椒类和花叶类的峰度较高，表明它们的数据分布更尖锐，而水生根茎类的偏度值最高，说明其数据分布更为右偏。数据整体不服从正态分布
- **变异系数：**花叶类具有最低的变异系数（0.48），表明具有相对较稳定的销售量；而水生根茎类具有最高的变异系数（0.849），表明销售量波动较大。

(3) 时序规律

我们对于三年内对每个月份的销量进行分类汇总，得到如下时序关系图：

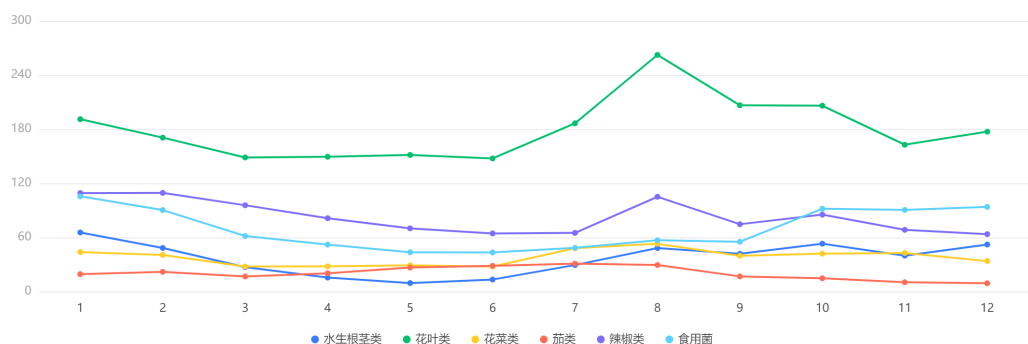


图2 每月的六大品类销量折线图

销量的**季节性规律**：茄类销售量波动不大，没有季节性规律。

除了茄类以外的品类的销量具有季节性规律，其中花叶类和辣椒类的销售量变化趋势类似，均在冬春季节销量较高，在8月份达到峰值。

我们对于每周一至每周日的销量进行分类汇总，得到如下时序关系图：

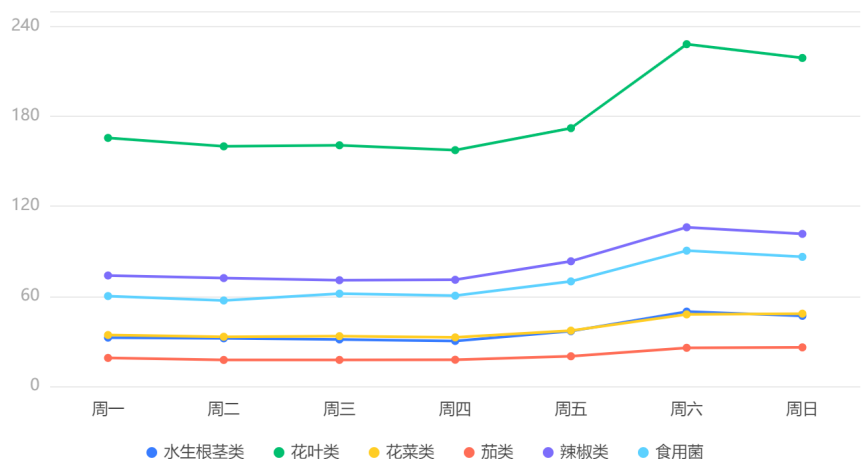


图3 每周的六大品类销量折线图

销量的**工作日-休息日规律**：茄类销售量波动不大，没有类似规律。

除了茄类以外的品类的销量均具有工作日-休息日规律，在工作日的销量较低，休息日的销量较高，且在周五开始呈现快速上升趋势，周六达到峰值，随后缓慢下降。

5.2.2 销售量的相互关系分析

(1) 灰色关联分析

要研究蔬菜各品类销售量的相互关系，即研究其销量的关联性。以每日总销量为母序列，六大品类的销售量为子序列，建立灰色关联分析模型。利用 SPSS 绘制灰色关联图像如下所示：

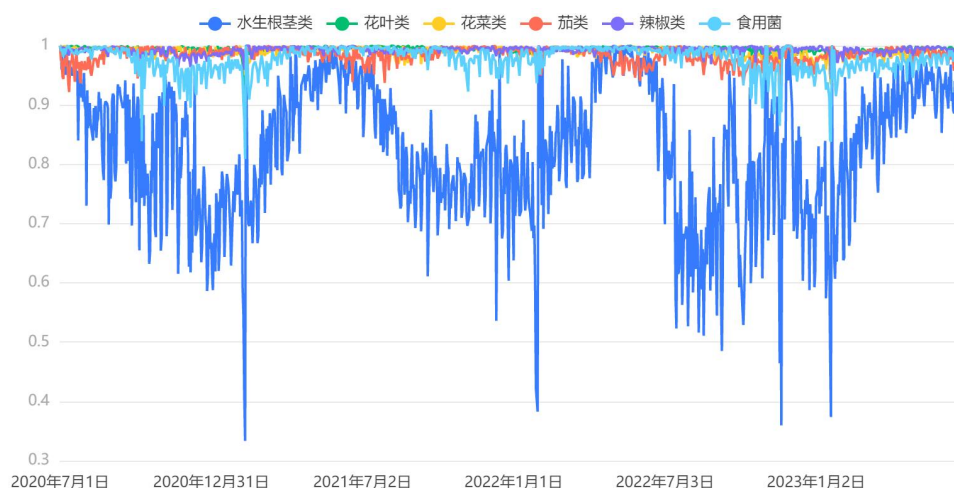


图 4 六大品类灰色关联度

从图中我们可以看出，各个子序列的分布比较集中，求解母序列和特征序列之间的灰色关联系数值，结合关联系数结果进行加权处理，最终得出关联度值，使用关联度值针对 6 个评价对象进行评价排序，得到结果如下：

表 3 六大品类的描述性统计分析结果

评价项	花叶类	辣椒类	花菜类	茄类	食用菌	水生根茎类
关联度	0.994	0.993	0.99	0.984	0.978	0.827

从上表可以看出：针对本次六个评价项，花叶类评价最高，关联度 0.994；其次是辣椒类，关联度为 0.993。六大品类关联度均大于 0.8，说明均与总销量有明显关联性。

(2) Spearman 相关性分析

从描述性统计中可知，销售量数据不服从正态分布，因此采用 Spearman 相关性分析，计算公式如下所示：

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (2)$$

其中，n 表示样本的数量，d 表示每一对排名的差异。利用 python 求解 6 个品类的 spearman 相关系数矩阵，结果如下图所示。

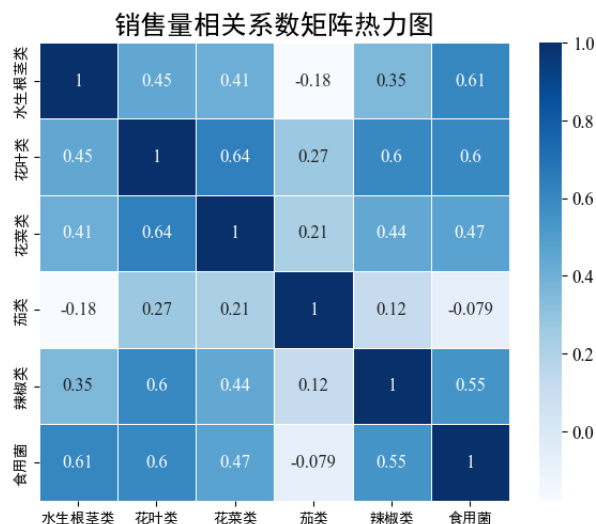


图 5 销售量相关系数矩阵热力图

由图可知：水生根茎类和食用菌、花叶类和辣椒类、花叶类和花菜类、食用菌和花叶类的销量的相关性较高，茄类和各个品类的相关性较低。据此可以推测出：相关性较高的两种产品在烹饪中具有互补性，因此常常被一起使用；茄类在烹饪和用途上与其他品类有较大的差异性，因此销量变化不大，没有季节性规律和工作日休息日规律，再一次验证了前文所总结的规律的合理性。

5.3 针对单品的分析

5.3.1 销售量的分布规律

我们对不同单品进行每日销售量的可视化，发现大部分单品呈现如下图所示的分布状况。

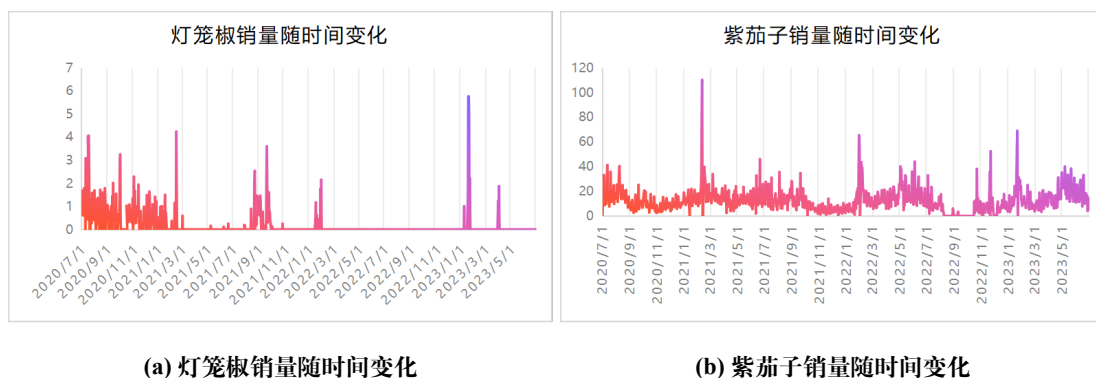


图 6 单品销售量随时间的变化趋势图

节假日激增规律：据上图可知，大部分单品在每年的 2 月份左右的销售量会呈现一

个激增的状态，即上图所示的高峰。分析可得每年的 2 月份为春节假期，销售量到达峰值。

5.3.2 销售量的相互关系

同样，由于销售数据不满足正态分布，我们采用 Spearman 相关性分析计算所有单品之间的相关系数。我们认为相关系数大于 0.7 或小于-0.7 的单品对具有强相关性，因此仅对此部分单品进行分析。根据商品对相关性的正负，我们将单品对分为互补品和替代品两种关系。

(1) 互补关系

当两种单品的相关性系数为正时，表明一种单品的需求增加，另一种单品的需求也随之增加。因此可将具有这种性质的单品对归类为互补品。

根据单品对是否属于同一品类，又可将互补品分为类内互补品和类间互补品。其中类内互补品表明两种单品用途相似但口感不同，人们倾向于同时购买他们。而类间互补品表明两种单品可以搭配使用。

表 4 部分互补品的展示

单品 1		单品 2		单品 1		单品 2	
单品 1		单品 2		单品 1		单品 2	
类内	海鲜菇 (2)	鲜木耳 (2)		类间	大白菜	杏鲍菇 (1)	
	姬菇 (2)	海鲜菇 (2)			枝江青梗散花	金针菇 (盒)	
	姜蒜小米椒组合装	红椒 (2)			杏鲍菇 (2)	螺丝椒 (份)	
	上海青 (份)	菜心 (份)			大白菜	金针菇 (1)	
	小皱皮 (份)	小米椒 (份)			枝江青梗散花	螺丝椒 (份)	

(2) 替代关系

当两种单品的相关性系数为负时，表明一种单品的需求增加，另一种单品的需求减少。因此可将具有这种性质的单品对归类为替代品。

根据单品对是否属于同一品类，又可将替代品分为类内替代品和类间替代品。其中类内替代品表明两种单品用途相似但口感也相似，人们倾向于只买其中一种。而类间替代品表明两种单品相互独立。

表 5 部分替代品的展示

	单品 1	单品 2		单品 1	单品 2
类内	青线椒	螺丝椒 (份)	类间	云南油麦菜	姜蒜小米椒组合装
	云南生菜 (份)	云南油麦菜		云南油麦菜	螺丝椒 (份)
	青线椒	姜蒜小米椒组合装		云南生菜 (份)	七彩椒 (2)
	青线椒	小皱皮 (份)		枝江青梗散花	大白菜

六、问题二的分析与解答

6.1 针对问题二第一小问

第一小问要求商超以品类为单位，根据成本加成定价法，探究品类销售总量与售价之间的关系。我们使用参数估计，通过探究成本加成定价模型机理，求出商超的固定成本和每一品类的单位可变成本。绘制售价与销量的散点图，发现大致呈负相关关系。随后通过非线性回归进行拟合，得到价格-需求的函数表达式。

6.1.1 成本加成定价模型

价格成本定价法基于实际成本，可以有效降低价格波动的幅度，减轻价格波动对商超的冲击^[1]。为分析其机理，本文构建成本加成定价模型：

$$s_i = \left(\frac{C_{i1}}{x_i} + C_{i2} \right) (1 + \alpha_i) \quad (3)$$

$$\alpha_i = \frac{s_i - b_i}{b_i} \cdot 100\% \quad (4)$$

其中， s_i 表示第 i 个品类的销售价格， x_i 表示补货量， α_i 表示加成率， b_i 为批发价格。 C_{i1} 表示总固定成本，包含商超的租金、工资、折旧等不随补货量增减而变化的成本，因此 C_{i1} 是一个固定的常数。 C_{i2} 表示单位变动成本，包括订货成本、运输成本、包装成本等随着补货量增减而变化的成本。加成率是指企业根据历史收益和成本制定的期望成本利润率，可以基于附件 2-3 提供的历史售价和批发价数据，计算每日平均加成率作为输入，使用时间序列预测，输出期望成本利润率的预测值。由上式可以得出，在总固定成本和单位变动成本保持不变的前提下，商品价格成本定价与补货量呈现负相关关系。

我们假定历史数据中商超处于供需平衡，即补货量 x_i 与销量 q_i 相等，代入每日历史销售量和售价，由拟合求得固定成本 $C_{i1} = 0.004$ 为一常量，每日的单位变动成本 C_{i2} 接近批发价。参数估计得到的 C_{i1} ， C_{i2} 用于后续商超补货和定价决策的分析。

6.1.2 价格-需求模型

为进一步研究销售总量与售价之间的关系，我们引入微观经济学中的价格-需求模型。通常情况下，商品的销售数量与价格呈现**负相关关系**，又由于商超蔬菜品类的销售现象较为复杂，故我们以销售量 q 为因变量，售价 s 为自变量，采用**非线性回归**的方式进行曲线拟合。在利用幂函数、对数函数、指数函数分别进行拟合比较不同的拟合优度后，针对 6 类品种，可得出销售量随售价变化的最优拟合曲线：

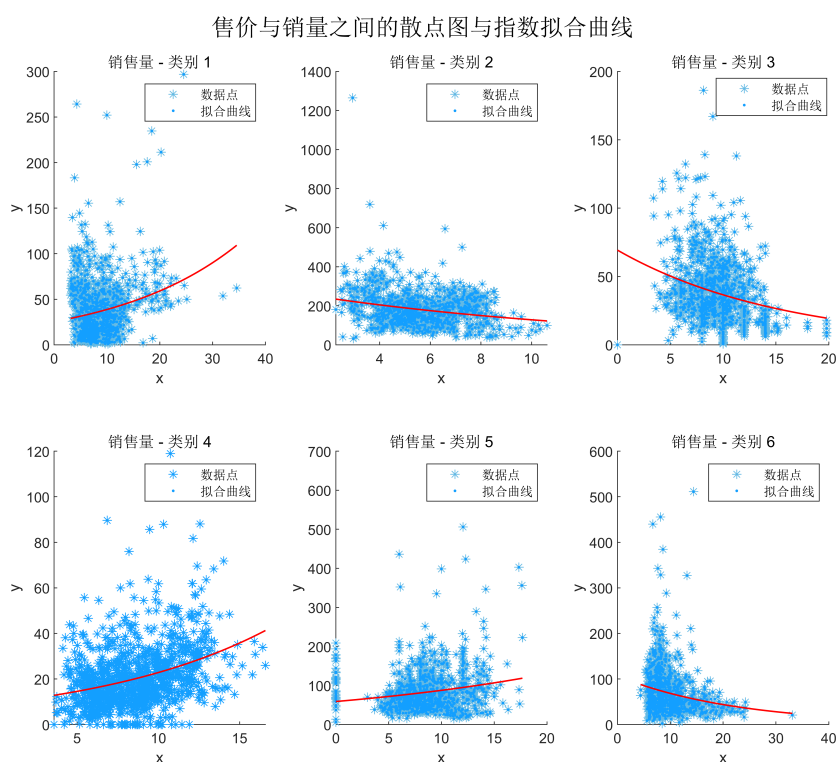


图 7 售价与销量之间的散点图与指数拟合曲线

以水生根茎类为例，我们最终选择如下最优指数函数作为拟合模型：

$$q = ae^{\frac{b}{s}} \quad (5)$$

通过最小二乘法得到各系数的参数估计值为 $a = 4.193, b = 20.527$ ，拟合优度高达 79.6%。因此，我们认为，本题商超的水生根茎类销售量与成本加成定价具有**指数相关性**，且为**负相关**。

6.2 针对问题二第二小问

题目要求以品类为单位，以商超收益最大化为目标制定未来一周的补货策略。首先我们基于历史销售数据，用时间序列模型预测该天的预计销量和预计批发价，并以预计销量的置信区间作为决策变量的约束条件。商超基于成本加成定价法制定售价，已知售价，可以用第一小问拟合得到的销售量与定价的关计算出该日实际销售量。已知补货量、计划售价、实际销售量、预计批发价，并结合历史数据和附件 4 的损耗率，即可构建商超的收益函数。最后，以计划补货量为决策变量，以收益最大化为目标函数，建立基于非线性规划的优化模型，并进行模型的检验验证合理性。

问题二规划模型的准备的示意图如下：

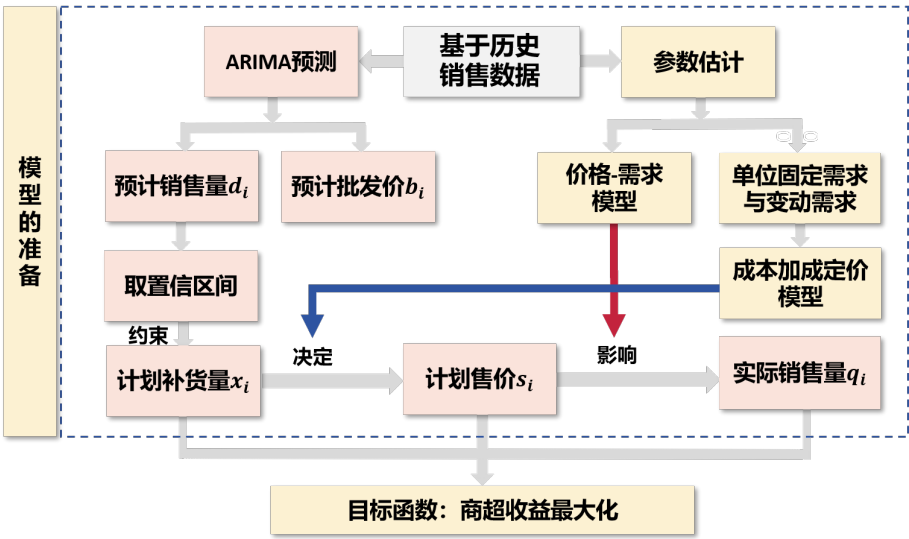


图 8 第二小问模型的准备示意图

6.2.1 菜品补货后的路径分析

商超对补货以后，菜品一般经过运输到达货架，运输过程中会有部分损耗，运损严重的蔬菜无法上架销售，其余蔬菜会上架。在针对某一天的上架蔬菜，由于蔬菜类商品保质期比较短，大部分品种无法隔日出售，商超会通过打折促进销量，多余部分直接浪费；少部分不易变质的菜品可以隔日再售。本文绘制菜品补货后的路径示意图如下所示：

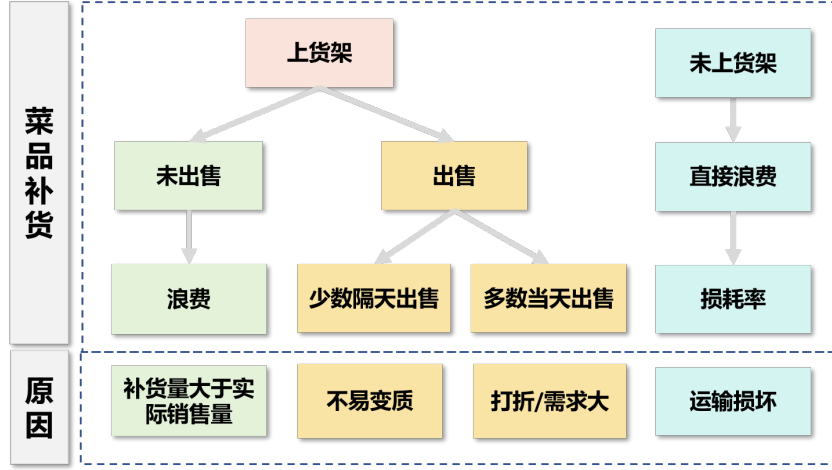


图9 针对某一天的菜品补货后的路径示意图

6.2.2 随机变量假设

假设第 i 品类的损耗率 k_i 服从正态分布，根据附件3可以求出六大品类的平均损耗率 μ_{k_i} 和标准差 σ_{k_i} ，本文考虑生成正态分布的随机量来模拟未来七天的菜品损耗率在均值附近的浮动，则有

$$k_i \sim N(\mu_{k_i}, \sigma_{k_i}) \quad (6)$$

引入随机变量假设的考虑在于：第一，每日品类内的菜品组合总是会有或多或少的差别，使得整体的平均损耗率在均值附近波动，因此引入随机变量假设有助于更好地反映实际情况。第二，随机量的引入使得未来七天的方案从具有平稳性到具有波动性，在丰富方案的同时，也具备了自动上下调整菜品补货量的能力，使模型具有更强的稳健性。

6.2.3 决策变量

我们为商超制定针对某一天的定价补货策略模型，策略包括定价和补货。而对于商超来说，定价可以由补货量通过成本加成定价方法计算得到，因此我们只需要以每日的不同品类蔬菜的补货量为决策变量，即 $x_i, i = 1, 2, \dots, 6$ 。

6.2.4 目标函数

商超利润可以分为品类收入与购买成本，本题需要以商超收益最大化为目标，因此目标函数为

$$\max \sum_{i=1}^6 \{(s_i d_i - b_i) q_i - k_i b_i x_i - b_i O_i [x_i (1 - k_i) - q_i]\} \quad (7)$$

其中， s_i 为成本加成定价， d_i 为折扣率， b_i 为批发价格， q_i 为实际销售量， k_i 为损耗率。根据题目定义大部分品种无法隔日销售，我们为了描述小部分品类出现隔天销售

的现象，引入 O_i 表示该商品是否出现隔天销售情况。为简化模型，我们假设隔天销售的品类不易变质且需求稳定，因而不存在打折销售和积压浪费的情况。

6.2.5 约束条件

(1) 等式约束: 针对第一小问销售总量与成本加成定价的关系，模型需要满足补货量-售价等式约束和售价-销售量等式约束，如下：

$$q_i = ae^{\frac{b}{s_i}} \quad (8)$$

$$s_i = \left(\frac{C_{i1}}{x_i} + C_{i2} \right) (1 + \alpha_i) \quad (9)$$

其中， $q_i = f(s_i)$ 由 6.2.2 中的价格-需求模型拟合得到，参数 C_{i1}, C_{i2} 可以由历史售价和历史销量数据通过均值估计求得。而在问题一中我们发现，菜品的销售量存在着时间波动趋势，而参数 α_i 是根据销售量和批发价求得的，据此，参数 α_i 需要通过**时间序列预测**得到。

(2) 供需约束: 第 i 个品类的平均损耗量 k_i 可通过附件 4 求出，商超为了满足市场对该类菜品的需求，需要满足该日进货未损耗的量大于每日实际销售量的约束，则有

$$x_i(1 - k_i) \geq q_i \quad (10)$$

(3) 补货量约束: 通过历史销量预测可以得到未来每日预计销量的点估计值，以 95% 的置信水平，求得置信区间的范围：置信区间 = 预测值 $\pm 1.96 \times$ 标准误差。将 $q_{\min}, q_{\max}$ 分别定义为置信区间的下界和上界，计划补货量即在该置信区间内波动，则有

$$q_{\min} \leq x_i \leq q_{\max} \quad (11)$$

(4) 折扣率约束: 引入变量 d_i 作为品类的折扣率，范围为 $[0, 1]$ ，当 $d_i = 1$ 时，表示该品类蔬菜不进行打折，当 $d_i < 1$ 时，表示该品类蔬菜进行打折。

(5) 当日销售约束: 引入 0-1 变量 O_i 来表示是否出现隔天销售情况。 $O_i = 1$ 表示该品类隔日则无法再售； $O_i = 0$ 表示该品类如果当日未售出，隔日仍可继续销售。

6.2.6 模型的小结

综上所述，我们可以建立如下最大化收益的非线性规划的优化模型：

$$\max \sum_{i=1}^6 \{(s_i d_i - b_i) q_i - k_i b_i x_i - b_i O_i [x_i (1 - k_i) - q_i]\} \quad (12)$$

$$\text{s.t.} \begin{cases} q_i = a e^{\frac{b}{s_i}} \\ s_i = \left(\frac{C_{i1}}{x_i} + C_{i2} \right) (1 + \alpha_i) \\ x_i (1 - k_i) \geq q_i \\ q_{\min} \leq x_i \leq q_{\max} \\ k_i \sim N(\mu_{k_i}, \sigma_{k_i}) \\ 0 < d_i \leq 1 \\ x_i \in N \\ O_i = 0, 1 \end{cases} \quad (13)$$

6.3 模型的求解

6.3.1 基于历史销量流水的预测

考虑到时间序列的趋势性和季节性，我们使用 ARIMA 模型对预计销售量、批发价格 b_i 、加成率 α_i 进行预测。对每日数据进行分类汇总，模型输入数据为历史数据每日的加权平均的销售量、批发价格以及利润率。其中，权重为当天的销售量除以总销售量。

在通过平稳性检验和白噪声检验以后，我们发现使用最近一年的数据拟合效果最好，拟合优度均大于 80%，说明模型性能优秀。预测结果图如下：

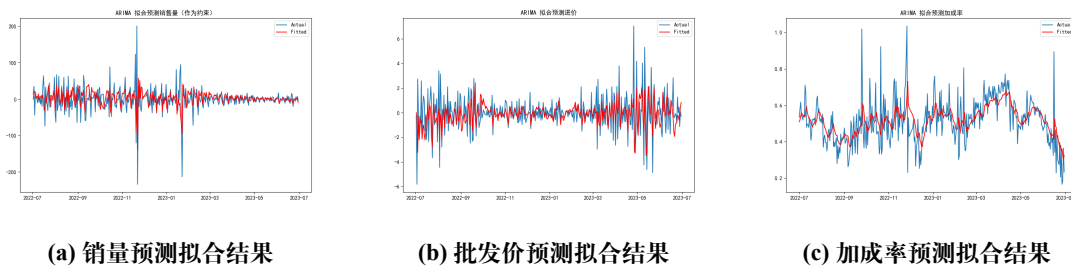


图 10 ARIMA 模型预测的结果图

6.3.2 遗传算法求解结果

对于该非线性的单目标优化类问题，显然可知目标函数是一个典型的非凸函数，因此我们选择遗传算法进行求解。遗传算法能够在全局搜索最优解，通过运用数学的方法

模拟自然界中的进化过程，将求解问题转化成类似自然界生物进化中的染色体基因的交叉、变异等过程。

我们通过使用 *python* 求解得到如下结果：

表 6 问题二的求解结果

		水生根茎类	花叶类	花菜类	茄类	辣椒类	食用菌
定价	7 月 1 日	6.077	5.050	11.636	5.608	7.895	14.927
	7 月 2 日	6.034	5.021	11.644	5.480	7.973	15.933
	7 月 3 日	6.027	5.020	11.652	5.460	7.949	15.716
	7 月 4 日	6.015	5.020	11.660	5.455	7.960	15.516
	7 月 5 日	6.004	5.020	11.668	5.452	7.960	15.647
	7 月 6 日	5.992	5.020	11.677	5.450	7.963	15.692
	7 月 7 日	5.981	5.020	11.685	5.447	7.965	15.651
补货量	7 月 1 日	25.47	149.03	22.42	21.89	80.29	48.15
	7 月 2 日	18.59	130.17	19.91	20.18	77.46	51.84
	7 月 3 日	11.25	113.17	18.78	16.75	74.32	53.43
	7 月 4 日	6.06	104.53	18.26	16.48	74.22	54.12
	7 月 5 日	10.12	113.84	18.02	16.71	77.67	54.44
	7 月 6 日	18.57	131.39	17.89	18.63	79.38	54.58
	7 月 7 日	26.54	144.39	17.81	19.23	79.71	54.66

6.4 模型的检验

(1) 灵敏度分析

我们对本模型的普适性进行分析。在实际情形中，进价会随着市场变化而上下波动。因此，本文进行灵敏度分析，对预测的批发价（即进价）进行上下 5% 的扰动，接着将波动以后的数据代入本题的优化模型中，结果如下：

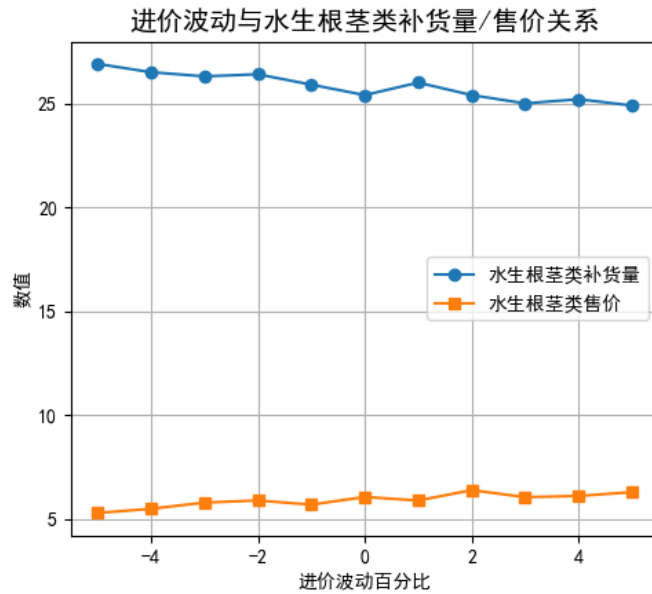


图 11 灵敏度分析检验模型普适性

由上图可知，我们的模型在面对批发价波动的时候，都能在较少的售价和补货量的波动下达到优秀的理想状态，可见模型的普适性较强。

(3) 遗传算法求解的可行性分析

我们使用的遗传算法考虑到自进化和交配，自进化能够加快算法的收敛，而交配的方式提高了算法的随机性，利于生成全局最优解。

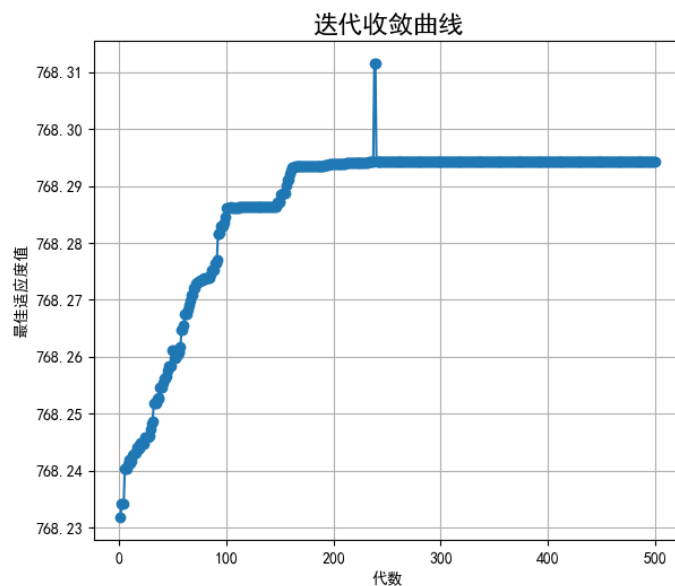


图 12 迭代收敛曲线

从图中可以清晰地看到随着迭代次数的增加，逐渐趋于稳定，在迭代 160 次左右时

可以找到近似最优解. 这说明遗传算法在不断地迭代过程中, 可以找到一个近乎稳定的最优解, 及其对应的优化运输方案。

七、问题三的分析与解答

7.1 问题三的思路分析

问题三与问题二是递进的关系, 新增了有限销售空间和最小陈列量的约束, 需要 we 们根据 2023 年 6 月 24-30 日的数据, 进一步制定商超在 2023 年 7 月 1 日单品的定价与补货策略。需要注意的是本题使用的数据时间跨度比较小, 属于小样本数据。与问题二不同的是, 问题三的单品数有 49 个, 且问题三要求尽量满足市场对各蔬菜单品的需求。为了方便表示销售单品数 [27, 33] 的限制, 本文引入 0-1 决策变量表示是否对该单品补货。同时, 为了尽量满足市场需求, 我们考虑在目标函数中引入惩罚项, 以增加模型的灵活性。

7.2 模型的建立

7.2.1 惩罚项的引入

在商超实际经营中, 通常无法利用历史销售和需求情况精确预测当天实际需求量, 商超希望最大化收益, 但也需要尽量满足市场各类蔬菜单品的需求。为了优化问题二的模型, 我们在目标函数中加入惩罚项:

$$\lambda \max(0, D_i) \quad (14)$$

其中, $D_i = q_i - y_i x_i (1 - k_i)$ 。惩罚系数 λ 用于权衡经济效益与市场需求。当 $y_i x_i (1 - k_i) < q_i$ 时, 此时, 未损耗的补货量无法满足市场需求, 需要在模型中加入惩罚项; 当 $y_i x_i (1 - k_i) \geq q_i$ 时, 此时, 未损耗的补货量能够充分满足市场需求, 惩罚项为 0。

加入惩罚项的考虑基于: 第一, 目标函数具有非凸性质, 存在多个局部最优解, 引入惩罚项可以帮助将问题转化为凸优化问题, 从而更有可能找到全局最优解, 而不会陷入局部最优解; 第二, 引入惩罚项可以增加灵活性, 通过调整惩罚项的大小 λ , 能够控制目标函数中约束违反的程度。这使得您可以根据问题的实际要求平衡目标函数的最大化和约束条件的满足。

7.2.2 基于单品互补替代关系的考虑

针对问题一的不同单品关联性分析, 我们发现部分单品之间存在互补或替代的联动效应, 不同单品之间的互补和替代关系会影响定价与补货策略。因此, 本文对问题一求

得的 *Spearman* 相关系数用欧几里得范数作归一化处理, 定义**归一化相关系数**为其余单品 (除第 i 种单品) 对基准补货量 x_i 的互补替代效应的度量

$$x'_i = x_i + r_{i1}x_1 + \cdots + r_{i,i-1}x_{i-1} + r_{i,i+1}x_{i+1} + \cdots + r_{i,49}x_{49} \quad (15)$$

最后对每一个单品的互补替代效应与基准补货量的乘积进行求和, 就可以得到第 i 个单品的实际补货量:

$$r_{ij} = \frac{\rho_{ij}}{\sqrt{\rho_{i1}^2 + \rho_{i2}^2 + \cdots + \rho_{i,i-1}^2 + \rho_{i,i+1}^2 + \cdots + \rho_{i,49}^2}} \quad (16)$$

其中, x'_i 为最终的单品补货量, x_i 为基准补货量, r_{ij} 为第 i 个单品和第 j 个单品的归一化相关系数, ρ_{ij} 为第 i 个单品和第 j 个单品的 *Spearman* 相关系数。

7.2.3 决策变量

与问题二类似, 除了引入单品补货量 $x_i, i = 1, 2, \dots, 49$ 作为决策变量以外, 还要考虑到部分单品不会进行补货操作, 因此, 我们还需要引入 0-1 决策变量 $y_i, i = 1, 2, \dots, 49$, 用来判断是否需要对第 i 种单品进行补货操作。当 $y_i = 1$ 时, 表示需要对第 i 种单品进行补货操作; 当 $y_i = 0$ 时, 表示不需要对第 i 种单品进行补货操作。

7.2.4 目标函数

我们需要以收益最大化作为商超的目标函数, 当 $D_i \geq 0$ 时, 此时无法满足市场需求, 需要考虑利用惩罚项扣除部分利润; 当 $D_i < 0$ 时, 此时未损耗的补货量大于需求量, 这里为了简化模型, 我们不考虑隔天销售的情况, 故此, 需要将利润扣除积压的补货量所造成的成本损失。

$$\max \sum_{i=1}^{49} \{[(s_i(1 - p_i + d_i p_i) - b_i)y_i q_i - x'_i y_i k_i b_i] - \lambda \max\{0, D_i\} - \max\{-b_i D_i, 0\} \quad (17)$$

其中, p_i 为折扣比例, x'_i 为实际补货量。其余参数均在问题二的模型建立中有说明, 故不予赘述。

7.2.5 约束条件

本题对问题二建立过的约束条件不作赘述, 仅介绍增加的约束条件。

(1) 销售单品总数约束: 限制可售单品总数在 27 – 33 个范围内, 则有

$$27 \leq \sum_{i=1}^{49} y_i \leq 33 \quad (18)$$

(2) 最小补货量约束：各单品订购量满足最小陈列量为 2.5 千克的约束，在这里，最小陈列量即为最小补货量，因此约束条件为：

$$x_i \geq 2.5 \quad (19)$$

(3) 其他约束：由于本题纳入了折扣比例 p_i ，因此需要增加折扣比例的约束，同时折扣率范围约束需要进行更新，最后决策变量 x_i 本身满足非负约束， y_i 为 0-1 变量。

$$0 < d_i < 1 \quad (20)$$

$$0 \leq p_i \leq 1 \quad (21)$$

$$x_i \in N, y_i = 0, 1 \quad (22)$$

7.2.6 模型的小结

综上所述，我们可以建立如下最大化收益的非线性混合整数规划的优化模型：

$$\begin{aligned} \max \sum_{i=1}^{49} \{[(s_i(1 - p_i + d_i p_i) - b_i]y_i q_i - x'_i y_i k_i b_i\} - \lambda \max\{0, D_i\} - \max\{-b_i D_i, 0\} \quad (23) \\ \text{s.t.} \begin{cases} q_i = a e^{\frac{b}{s_i}} \\ s_i = (\frac{C_{i1}}{x'_i} + C_{i2})(1 + \alpha_i) \\ x'_i = x_i + r_{i1}x_1 + \cdots + r_{i,i-1}x_{i-1} + r_{i,i+1}x_{i+1} + \cdots + r_{i,49}x_{49} \\ 27 \leq \sum_{i=1}^{49} y_i \leq 33 \\ x_i \geq 2.5 \\ q_{\min} \leq x_i \leq q_{\max} \\ 0 < d_i < 1 \\ 0 \leq p_i \leq 1 \\ x_i \in N \\ y_i = 0, 1 \end{cases} \quad (24) \end{aligned}$$

7.3 模型的求解

7.3.1 灰色预测

因为本题时间序列数据具有信息不确定、样本规模 (7×49) 小、预测趋势短的特点，故本题更适合使用灰色预测 $GM(1, 1)$ 模型预测批发价格、销售数量和加成率。

(1) 缺失值的填充

使用灰色预测模型预测前，需要汇总 2023 年 6 月 24-30 日每日的销售量和批发价格数据。然而根据 2023 年 6 月 24-30 日的附件 2 流水销售数据，发现可售的 49 种单品并非每一天都含有销售数据。对比附件 3 可知，缺失值包含已进货但销售量为 0，和未进货两种状态，因此我们对销售量和批发价格按下图所示进行填充。

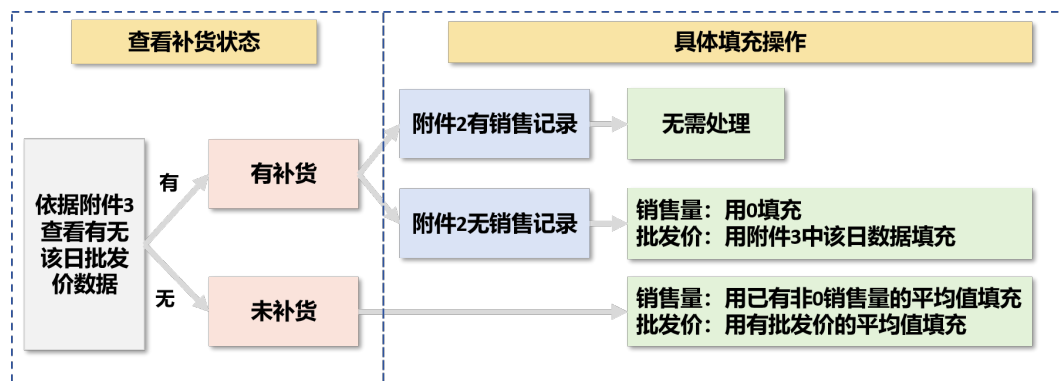


图 13 缺失值填充的操作示意图

(2) 级比检验

在进行灰色预测前，需要先进行级比检验，以判断适合构建灰色预测模型。以预测小米椒的销售量为例，级比检验结果表如下所示：

表 7 级比检验结果表

日期	原始值	级比值	平移转换后序列值	平移转换后级比值
2023/6/24	20	-	86	-
2023/6/25	17	1.176	83	1.036
2023/6/26	20	0.85	86	0.965
2023/6/27	19	1.053	85	1.012
2023/6/28	18	1.056	84	1.012
2023/6/29	33	0.545	99	0.848
2023/6/30	23	1.435	89	1.112

从上表分析可以得到，平移转换后序列的所有级比值都位于区间 (0.779, 1.284) 内，说明平移转换后序列适合构建灰色预测模型。

(3) 预测结果

根据 $GM(1, 1)$ 模型的思想，首先对销售量数据进行累加操作，构建一阶线性差分方程 $X(k) + aY(k) = B$ ，其中 $X(k)$ 为累加数据， $Y(k)$ 为原始数据。代入经过缺失值

处理后的销售量数据分别进行参数估计后，可以求得发展系数 $a = -0.022$ ，灰色作用量 $b = 80.065$ ，后验差比 $c = 0.46$ ，最终得到小米椒在 7 月 1 日预测的未来销售量 q_i 为 27.65。预测的未来销售量通过取置信区间作为约束条件，加入模型的求解中。同理，通过灰色预测求得的未来批发价格为 2.11 元。

7.3.2 求解结果

使用 *python* 对混合整数非线性规划问题进行求解，我们求得当商超实现收益最大化时，一共需要补货 33 种单品，以下是前 8 类单品的补货和定价策略，其他单品结果见附录。

表 8 问题三的求解结果（部分）

单品名称	补货量	不打折价格	打折价格	打折比例
小米椒 (份)	23	5.7743	4.105	0.0205
西兰花	16.9	12.3943	7.59	0
西峡花菇 (1)	6.572	24	15.6	0
七彩椒 (2)	7.036	20.5517	14.2433	0.1851
鲜木耳 (份)	9.7	6.3	2.425	0
长线茄	8.789	12.0171	7	0
枝江青梗散花	11.187	12.785	8.8506	0.1215
紫茄子 (2)	14.365	5.6829	3.43	0

以小米椒 (份) 为例，2023 年 7 月 1 日的补货策略为：共需要补货 23 千克，定价策略为：原价 5.7743 元/千克，并**设定一个实时的反馈机制**，根据当天的预测销售量 27.65 千克，当销售数量达到预测销售量的 2.05% 时，开始进行打折，打折后价格为 4.105 元/千克。其他单品作同样补货和定价策略。

八、问题四的分析与解答

8.1 问题四的思路分析

针对问题四，题目要求我们采集更多相关数据，帮助商超制定蔬菜商品的补货和定价策略，以实现收益最大化的目标。影响补货和定价决策的因素除了固定成本、变动成

本、损耗率等因素之外，还可以从竞争冲击、供应链费用、消费者反馈、政策干预等多方面进行挖掘。

8.2 针对竞争冲击

8.2.1 数据的采集

在互联网大数据迅速发展的背景下，线上销售渠道逐渐兴起，并对商超线下销售产生冲击，与线下商超在菜品销售中形成竞争关系。我们根据本文所给的菜品类型进行检索，在线上美团买菜 APP 和叮咚买菜 APP 上统计该类菜品的价格。利用线上菜品的售价和线下菜品的售价，计算线上溢价率。以下是部分菜品的线上溢价率统计图：

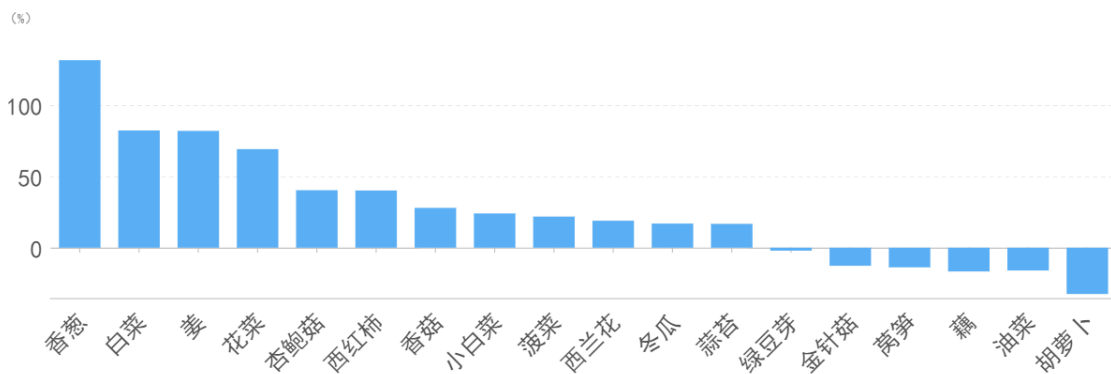


图 14 不同菜品的线上溢价率

8.2.2 模型的优化

我们将线上溢价率设为参数 e_i 纳入问题三建立的优化模型中，目标函数更新为：

$$\max \sum_{i=1}^{49} \{[(s_i(1 - p'_i + d_i p'_i) - b_i)y_i q_i - x'_i y_i k_i b_i] - \lambda \max\{0, D_i\} - \max\{-b_i D_i, 0\} \quad (25)$$

其中， $p'_i = p_i(1 - e_i)$ 。

同样，利用问题三的算法对模型进行求解，以西兰花为例，求得西兰花的补货量为 25.1 千克，售价为 9.24 元/千克。显然，与通过问题三求得的西兰花补货量和售价明显不同，可见考虑线上平台的冲击能帮助商超更好地制定蔬菜商品的补货和定价决策。因此，商超在进行补货和定价决策时，要考虑到线上菜品的价格变动，并由此延申，关注线上菜品的运输便利性、品质新鲜度等因素。

8.3 针对供应链费用

供应链费用会直接影响到蔬菜的成本结构，进而影响到企业的最终利润。商超可以收集运输成本、库存持有成本和订单处理成本数据，通过尽可能减小这些供应链费用，

来实现商超利润最大化的目标。

为了最大化利润并降低供应链费用，商超可以采取以下措施：

(1) **库存管理**：基于库存数据，优化库存水平和补货策略，以减少库存持有成本。使用定量模型来预测需求，以确保库存水平充足但不过剩。

(2) **运输优化**：基于运输数据，优化运输方式和路线，以降低运输成本。合并订单或选择更经济高效的运输方式可以减少费用。

(3) **订单处理改进**：通过减少订单处理成本，提高订单处理效率，例如自动化订单处理流程，可以降低相关费用。

8.4 针对消费者反馈

消费者反馈可以提供关于产品质量、价格敏感度、需求变化等方面的信息：如果消费者对某些产品不满意，商超可能需要重新考虑供应商选择和库存管理；如果消费者普遍认为价格太高，商超可能需要重新考虑定价策略，以更吸引顾客；如果消费者反映对某种蔬菜有较高需求，商超可以考虑增加库存量。因此商超可以收集消费者满意度调查、销售数据和消费者评论和投诉数据。基于以上数据实现：

(1) **产品质量改进**：根据消费者满意度调查和反馈，商超可以采取改进产品质量，确保产品符合消费者期望。

(2) **价格策略调整**：如果消费者反馈价格太高，商超可以考虑重新定价或提供折扣以吸引更多顾客。

(3) **产品选择和定价**：基于市场需求和竞争对手的数据，商超可以调整产品组合和定价策略，以满足消费者的需求。

8.5 针对政策干预

根据三年内单品销售量随时间分布的趋势图可以看出，2022 年 11 月出现波峰。通过查找资料，我们发现政府在这一端时间内鼓励“菜篮子”政策的实施。据此，我们推测政策干预能够通过影响销售量，进一步影响补货和定价决策。因此，商超可以收集政府鼓励蔬菜的政策，估计政策对销售量的影响。基于以上数据有利于：

(1) **风险管理**：随着政策的改变，市场环境可能会发生变化，可能对销售量和市场需求产生不确定性。通过估计政策的冲击，商超可以更好地了解潜在的风险，采取适当的风险管理策略，以减少不确定性带来的负面影响。

(2) **市场份额保护**：如果政策变化可能导致市场份额的损失，估计政策的冲击可以帮助商超采取措施来保护其市场份额。商超可以调整定价、促销活动等策略，以吸引更多的顾客并维持市场份额。

九、模型的评价与改进

9.1 模型的优点

(1) 每日品类内的菜品组合总是会有或多或少的差别，使得整体的平均损耗率在均值附近波动，因此引入随机变量假设有助于更好地反映实际情况。此外，随机量的引入使得未来七天的方案抛弃了一成不变的平稳型，从而具有波动性，在丰富方案的同时，也具备了自动上下调整菜品补货量的能力，使模型具有更强的稳健性。

(2) 本文加入惩罚项的考虑基于：第一，目标函数具有非凸性质，存在多个局部最优解，引入惩罚项可以帮助将问题转化为凸优化问题，从而更有可能找到全局最优解，而不会陷入局部最优解；第二，引入惩罚项可以增加灵活性，通过调整惩罚项的大小 λ ，能够控制目标函数中约束违反的程度。这使得您可以根据问题的实际要求平衡目标函数的最大化和约束条件的满足。

9.2 模型的缺点及改进方向

(1) 由于根据附件中的数据，退货数量可以和销售数量正负抵消，因此在非线性规划的优化模型中未将退货项纳入模型，但在实际情况中，退货会产生一定的运营成本，会导致用户信誉下降，间接导致企业收益减小。

(2) 针对小部分可以隔天销售的品种，本文假设商超不会因为该品种库存积压而产生损失，但在实际情况中，隔天销售的品种较为复杂，当日之前所有时间的未销售数量存在打折、浪费、库存等情况，可以在后续的模型中进行改进。

参考文献

- [1] 吴德庆. 管理经济学. 北京: 中国人民大学出版社, 1999: 130-131
- [2] 毛莉莎. 供应链视角下蔬菜批发市场定价策略及产销模式研究 [D]. 中南林业科技大学, 2023.
- [3] 安琪. 基于成本加成定价法的科技查新服务定价研究 [J]. 图书馆研究与工作, 2021(10): 25-31+24.
- [4] 陈龙, 林茂仙. 基于优化的成本加成商业银行贷款利率定价 [J]. 商场现代化, 2021(17): 113-115.
- [5] 张辉锋, 景恬. 成本加成与消费者感知价值的结合: 知识付费产品的定价模型 [J]. 新闻与传播研究, 2021, 28(01): 38-51+127.
- [6] 王珊珊. 农贸市场蔬菜零售价格形成问题研究 [D]. 华中农业大学, 2014.
- [7] 崔利刚, 李亚丽, 刘锦杏等. 考虑保鲜技术投资的多品补货与定价联合决策 [J]. 工业工程与管理, 2023, 28(03): 17-26.
- [8] 李亚丽. 考虑保鲜技术投资的联合补货与定价协同决策 [D]. 重庆交通大学, 2023.

附录

表 9 问题一相关系数表汇总（互补替代关系分析）

单品 1	1 类别	单品 2	2 类别	相关系数
红橡叶	花叶类	红珊瑚 (粗叶)	花叶类	1.00
红橡叶	花叶类	绿牛油	花叶类	1.00
红珊瑚 (粗叶)	花叶类	红珊瑚 (粗叶)	花叶类	1.00
红珊瑚 (粗叶)	花叶类	绿牛油	花叶类	1.00
鲜木耳 (2)	食用菌	鲜木耳 (份)	食用菌	0.99
小青菜 (2)	花叶类	枝江红菜苔 (份)	花叶类	0.91
杏鲍菇 (1)	食用菌	金针菇 (1)	食用菌	0.90
七彩椒 (2)	辣椒类	上海青	花叶类	0.88
小皱皮 (份)	辣椒类	小米椒 (份)	辣椒类	0.87
金针菇 (1)	食用菌	青梗散花	花菜类	0.87
姜蒜小米椒组合装 (小份)	辣椒类	螺丝椒 (份)	辣椒类	0.86
大白菜	花叶类	金针菇 (1)	食用菌	0.86
上海青 (份)	花叶类	菜心 (份)	花叶类	0.85
红椒 (2)	辣椒类	螺丝椒 (份)	辣椒类	0.85
小皱皮 (份)	辣椒类	螺丝椒 (份)	辣椒类	0.84
云南生菜 (份)	花叶类	小米椒 (份)	辣椒类	0.84
枝江青梗散花	花菜类	螺丝椒 (份)	辣椒类	0.84
姬菇 (1)	食用菌	海鲜菇 (1)	食用菌	0.84
姜蒜小米椒组合装 (小份)	辣椒类	小皱皮 (份)	辣椒类	0.83
杏鲍菇 (2)	食用菌	枝江青梗散花	花菜类	0.83
云南生菜 (份)	花叶类	小皱皮 (份)	辣椒类	0.82
云南生菜 (份)	花叶类	姜蒜小米椒组合装 (小份)	辣椒类	0.82
小米椒 (份)	辣椒类	螺丝椒 (份)	辣椒类	0.82
大白菜	花叶类	青梗散花	花菜类	0.82
杏鲍菇 (1)	食用菌	海鲜菇 (1)	食用菌	0.82
杏鲍菇 (1)	食用菌	青梗散花	花菜类	0.82
姬菇 (2)	食用菌	海鲜菇 (2)	食用菌	0.82
姜蒜小米椒组合装 (小份)	辣椒类	枝江青梗散花	花菜类	0.81
海鲜菇 (2)	食用菌	鱼腥草	花叶类	0.81
枝江青梗散花	花菜类	金针菇 (盒)	食用菌	0.81
小皱皮 (份)	辣椒类	枝江青梗散花	花菜类	0.81

表 9 – 续页

单品 1	1 类别	单品 2	2 类别	相关系数
姜蒜小米椒组合装 (小份)	辣椒类	红椒 (2)	辣椒类	0.81
云南生菜 (份)	花叶类	枝江青梗散花	花菜类	0.81
海鲜菇 (2)	食用菌	鲜木耳 (2)	食用菌	0.81
云南生菜 (份)	花叶类	螺丝椒 (份)	辣椒类	0.81
姜蒜小米椒组合装 (小份)	辣椒类	小米椒 (份)	辣椒类	0.80
大白菜	花叶类	杏鲍菇 (1)	食用菌	0.80
红椒 (1)	辣椒类	螺丝椒 (份)	辣椒类	-0.70
云南油麦菜	花叶类	姜蒜小米椒组合装 (小份)	辣椒类	-0.71
小皱皮 (份)	辣椒类	青线椒	辣椒类	-0.71
姜蒜小米椒组合装 (小份)	辣椒类	青线椒	辣椒类	-0.71
云南油麦菜	花叶类	螺丝椒 (份)	辣椒类	-0.72
云南生菜	花叶类	云南生菜 (份)	花叶类	-0.73
云南油麦菜	花叶类	云南生菜 (份)	花叶类	-0.74
螺丝椒 (份)	辣椒类	青线椒	辣椒类	-0.74

表 10 问题三结果汇总

单品名称	进货量	不打折价格	打折价格	打折比例
小米椒 (份)	23	5.774285714	4.105	0.020496894
西兰花	16.9	12.39428571	7.59	0
西峡花菇 (1)	6.572	24	15.6	0
七彩椒 (2)	7.036	20.55166667	14.24333333	0.185119048
鲜木耳 (份)	9.7	6.3	2.425	0
长线茄	8.789	12.01714286	7	0
枝江青梗散花	11.187	12.785	8.850555556	0.121527778
紫茄子 (2)	14.365	5.682857143	3.43	0
螺丝椒	8.47	12.88285714	8.97	0
云南油麦菜 (份)	25	4.475714286	3.132857143	0.266379135
云南生菜 (份)	36	4.571428571	3.380816327	0.206943132
芜湖青椒 (1)	15.896	5.445714286	3.63	0
上海青	7.036	7.968571429	4.09	0
净藕 (1)	6.44	15.63428571	9.960816327	0.29037474
姜蒜小米椒组合装 (小份)	9	4.692857143	3.67	0.12414966

表 10 – 续页

单品名称	进货量	不打折价格	打折价格	打折比例
奶白菜	9.044	4.831666667	2.56	0
竹叶菜	12.884	3.574285714	2.15	0
螺丝椒 (份)	10	6.055714286	3.94	0.036090226
菱角	3.396	13.98666667	9.16	0
双孢菇 (盒)	9	5.507142857	3.471428571	0.242746936
菠菜 (份)	11	6.572	4	0.486666667
红薯尖	5.88	4.767142857	2.6	0
木耳菜	5.185	5.908571429	4.044	0.275396825
娃娃菜	6	6.468571429	4.666	0.155058515
小青菜 (1)	4.709	5.07	2.7	0
小皱皮 (份)	10	3.357142857	2.462244898	0.213827839
洪湖藕带	4.339	26	14.65892256	0.468367347
青红杭椒组合装 (份)	4	5.91	3.42	0.071428571
云南生菜	2.904	10.145	6.68	0
苋菜	6.726	3.628571429	2.21	0
高瓜 (1)	3.78	16.13	11.02240741	0.483194187
金针菇 (盒)	13	1.997142857	1.224666667	0.181623932
海鲜菇 (包)	6	2.995714286	1.956	0.273726274

Listing 1. 问题一

```

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 数据表连接
# 读取附件1和附件2的数据
fujian1 = pd.read_excel('附件1.xlsx')
fujian2 = pd.read_excel('附件2.xlsx')

# 使用merge函数将两个数据框连接起来，根据单品编码进行连接
result = pd.merge(fujian2, fujian1[['单品编码', '单品名称', '分类名称']], on='单品编码',
                  how='left')

result.to_excel('附件2_带单品名称和分类名称.xlsx', index=False)

```

```

#连接附件2和附件3
# 读取附件1和附件2的数据
fujian1 = pd.read_excel('附件1.xlsx')
fujian2 = pd.read_excel('附件2.xlsx')

# 获取附件1和附件2中的单品编码
product_codes_fujian1 = set(fujian1['单品编码'])
product_codes_fujian2 = set(fujian2['单品编码'])

# 找出在附件1中出现但在附件2中没有出现的单品编码
missing_product_codes = product_codes_fujian1 - product_codes_fujian2

# 根据缺失的单品编码从附件1中筛选相应的单品名称和分类名称
missing_products_info = fujian1[fujian1['单品编码'].isin(missing_product_codes)][['单品编码',
    '单品名称', '分类名称']]

# 输出缺失的商品信息
print("以下是在附件1中出现但在附件2中没有出现的商品信息：")
print(missing_products_info)

# 读取附件2_带单品名称和分类名称和附件3的数据
fujian2 = pd.read_excel('附件2_带单品名称和分类名称.xlsx')
df3 = pd.read_excel('附件3.xlsx')

# 使用merge函数进行左连接，根据销售日期和单品编码连接，保留附件2的所有数据
result = pd.merge(fujian2, df3[['日期', '单品编码', '批发价格(元/千克)']], left_on=['销售日期',
    '单品编码'], right_on=['日期', '单品编码'], how='left')

# 删除连接后多余的日期列
result.drop(columns=['日期'], inplace=True)

# 将结果中的NaN值（没有匹配的批发价格）替换为空白字符串
result['批发价格(元/千克)'].fillna('', inplace=True)

# 将结果保存到一个新的Excel文件中
result.to_excel('附件2_带单品名称分类名称和批发价格.xlsx', index=False)

# 单品销量汇总
# 读取附件2_带单品名称分类名称和批发价格的数据
df = pd.read_excel('附件2_带单品名称分类名称和批发价格.xlsx')

# 将销售日期转换为日期格式，并提取年月日
df['销售日期'] = pd.to_datetime(df['销售日期']).dt.date

```

```

# 按销售日期和单品编码分组，累加销量
grouped = df.groupby(['销售日期', '单品编码', '单品名称', '分类名称'],
                      as_index=False)['销量(千克)'].sum()

# 将单品编码转为文本格式，以避免科学计数法
grouped['单品编码'] = grouped['单品编码'].astype(str)

# 保存结果到一个新的Excel文件
grouped.to_excel('销售汇总表.xlsx', index=False)

# 补全日期序列
# 读取水生根茎类数据文件
df = pd.read_csv("六类单日.csv", encoding='gbk')

# 创建一个包含完整日期范围的日期列
start_date = pd.to_datetime('2020-07-01')
end_date = pd.to_datetime('2023-06-30')
date_range = pd.date_range(start_date, end_date, freq='D')
date_df = pd.DataFrame({'销售日期': date_range})

# 将原始数据表中的"销售日期"列转换为datetime64[ns]格式
df['销售日期'] = pd.to_datetime(df['销售日期'])

# 合并数据表并填充缺失值为0
merged_df = date_df.merge(df, on='销售日期', how='left').fillna({'食用菌': 0})

merged_df.to_csv('填充后.csv', index=False)

# 异常值处理

# 异常值处理1：删除利润率大于75%的数据
# 读取包含数据的文件，使用实际的文件名
df = pd.read_excel('附件2_带单品名称分类名称和批发价格.xlsx')

# 计算利润率
df['利润率'] = (df['销售单价(元/千克)'] - df['批发价格(元/千克)']) / df['销售单价(元/千克)']

# 筛选出利润率不大于0.75的记录
yichang1 = df[df['利润率'] <= 0.856]

# 将销售日期列转换为日期时间类型
yichang1['销售日期'] = pd.to_datetime(yichang1['销售日期'])
# 保持日期列的格式不变，保留年月日，不包含时分秒

```



```

yichang1['销售日期'] = yichang1['销售日期'].dt.strftime('%Y-%m-%d')

# 将单品编码列转换为文本格式，避免科学计数法
yichang1['单品编码'] = yichang1['单品编码'].astype(str)

# 将筛选后的数据保存到一个新文件，假设文件名为filtered_data.csv
yichang1.to_csv('异常值处理1.csv', index=False)

#

# 读取包含数据的Excel文件，假设文件名为异常值处理1.xlsx，你需要将文件名替换成实际的文件名
df = pd.read_csv('异常值处理1.csv')

# 筛选出是否打折销售为"否"且销售单价小于批发价格的记录
yichang2 = df[((df['是否打折销售'] == '否') & (df['销售单价(元/千克)'] >=
    df['批发价格(元/千克)']))|(df['是否打折销售'] == '是')]

yichang2['销售日期'] = pd.to_datetime(yichang2['销售日期'])
# 保持日期列的格式不变，保留年月日，不包含时分秒
yichang2['销售日期'] = yichang2['销售日期'].dt.strftime('%Y-%m-%d')

# 将单品编码列转换为文本格式，避免科学计数法
yichang2['单品编码'] = yichang2['单品编码'].astype(str)

# 将删除后的数据保存到一个新Excel文件，假设文件名为filtered_data.xlsx
yichang2.to_csv('异常值处理2.csv', index=False)

#异常值处理3
# 读取 Excel 文件
yichang3 = pd.read_csv('异常值处理2.csv',encoding='utf-8')

# 定义一个函数来筛选符合条件的行
def filter_group(group):
    # 找到是否打折销售为是的行
    discount_sales = group[group['是否打折销售'] == '是']

    if not discount_sales.empty:
        # 找到组内非打折销售为否的最低销售单价
        min_price = group[group['是否打折销售'] == '否']['销售单价(元/千克)'].min()

        # 筛选出满足条件的行
        return discount_sales[discount_sales['销售单价(元/千克)'] > min_price]
    else:
        # 如果该分组没有打折销售为是的行，则返回整个分组
        return

```

```

# 根据销售日期和单品名称进行分组，然后应用筛选函数
filtered_data = yichang3.groupby(['销售日期',
    '单品名称']).apply(filter_group).reset_index(drop=True)
# 删除满足条件的行
yichang3 = yichang3.drop(filtered_data.index)

yichang3['销售日期'] = pd.to_datetime(yichang3['销售日期'])
# 保持日期列的格式不变，保留年月日，不包含时分秒
yichang3['销售日期'] = yichang3['销售日期'].dt.strftime('%Y-%m-%d')

# 将单品编码列转换为文本格式，避免科学计数法
yichang3['单品编码'] = yichang3['单品编码'].astype(str)

# 将筛选后的结果保存到新的 Excel 文件
yichang3.to_csv('异常值处理3.csv', index=False)

import pandas as pd
# 单日单品汇总
# 读取数据
data = pd.read_csv('单品单日.csv', encoding='gbk')

# 创建一个新的DataFrame来保存转换后的数据
new_data = pd.DataFrame()

# 遍历每个单品名称
unique_products = data['单品名称'].unique()
for product in unique_products:
    # 从原始数据中选择特定单品名称的记录
    product_data = data[data['单品名称'] == product]
    print(product_data)

    # 获取销售日期和销售量的列，并重命名列名
    product_data = product_data.rename(columns={'销售日期': f'{product}销售日期', '销量(千克)':
        f'{product}销售量'})

    # 将这些列合并到新的DataFrame中
    new_data = pd.concat([new_data, product_data[[f'{product}销售日期', f'{product}销售量']],
        axis=1)

# 保存转换后的数据到新的CSV文件
new_data.to_csv('单日单品汇总.csv', index=False, encoding='utf-8')

# 读取包含多个类别销售数据的文件
df = pd.read_csv("单日单品汇总.csv", encoding='gbk')

```

```

# 创建一个包含完整日期范围的日期列
start_date = pd.to_datetime('2020-07-01')
end_date = pd.to_datetime('2023-06-30')
date_range = pd.date_range(start_date, end_date, freq='D')
date_df = pd.DataFrame({'销售日期': date_range})

# 创建一个空的DataFrame用于存储最终结果
final_df = date_df.copy()

# 获取每个类别的销售日期列和销售量列，并合并到最终DataFrame
categories = df.columns[::2] # 假设每两列为一组，第一列是销售日期，第二列是销售量
for category in categories:
    date_column = category
    sales_column = category.replace("销售日期", "销售量")

    # 提取类别名称，假设列名格式为"类别名称销售日期"
    category_name = category.split("销售日期")[0]

    # 将原始数据表中的销售日期列转换为datetime64[ns]格式
    df[date_column] = pd.to_datetime(df[date_column])

    # 合并数据并填充缺失值
    merged_df = date_df.merge(df[[date_column, sales_column]], left_on='销售日期',
                              right_on=date_column,
                              how='left').fillna({sales_column: 0})

    # 重命名销售量列，以便区分不同类别
    merged_df = merged_df.rename(columns={sales_column: f'销售量(千克)_{category_name}'})

    # 合并结果到最终DataFrame
    final_df = final_df.merge(merged_df[['销售日期', f'销售量(千克)_{category_name}']],
                              on='销售日期', how='left')

# 输出结果到CSV文件
final_df.to_csv('填充后.csv', index=False)

# 读取填充后的CSV文件
data = pd.read_csv('填充后.csv', encoding='utf-8')

# 删除销售日期列中的重复日期行
data = data.drop_duplicates(subset=['销售日期'])

# 保存修改后的数据到新的CSV文件
data.to_csv('填充后单品单日汇总.csv', index=False, encoding='utf-8')

```

```

# 六大品类相关性矩阵
# 修改matplotlib字体设置，支持中文显示 (Windows)
plt.rcParams['font.sans-serif'] = ['SimHei'] # 或者 ['Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 读取包含销量数据的CSV文件
df = pd.read_csv("填充后六类单日汇总.csv", encoding='gbk')

# 选择包含销售量数据的列
selected_columns = ['水生根茎类', '花叶类', '花菜类', '茄类', '辣椒类', '食用菌']

# 选择这些列的数据
selected_data = df[selected_columns]

# 计算相关系数矩阵
correlation_matrix = selected_data.corr(method='spearman')

# 绘制热力图，使用高科技感的颜色
cmap = sns.color_palette("Blues", as_cmap=True) # 选择深蓝色系

# 设置字体系列为Times New Roman
annot_kws = {'family': 'Times New Roman', 'size': 12, 'weight': 'normal'}

ax = sns.heatmap(correlation_matrix, annot=True, cmap=cmap, linewidths=.5, annot_kws=annot_kws)

# 获取颜色条对象
cbar = ax.collections[0].colorbar

# 设置颜色条上标签的字体系列和大小
for label in cbar.ax.get_yticklabels():
    label.set_family('Times New Roman')
    label.set_fontsize(12)

# 计算注释文本的高度和宽度，以确定画布大小
height, width = correlation_matrix.shape

# 设置 DPI 值，增加分辨率和清晰度
plt.gcf().set_size_inches(0.2 * width, 0.2 * height)
plt.gcf().set_dpi(1200) # 设置 DPI

plt.style.use("ggplot")
plt.title('销售量相关系数矩阵热力图', fontsize=18)
plt.show()

```

```

# 读取填充后的CSV文件
data = pd.read_csv('填充后单品单日汇总.csv', encoding='utf-8')

# 提取除第一列之外的所有列
data_numeric = data.iloc[:, 1:]

# 计算Spearman相关系数
spearman_corr = data_numeric.corr(method='spearman')

# 绘制相关系数矩阵图
plt.figure(figsize=(12, 8))
sns.heatmap(spearman_corr, annot=True, cmap='coolwarm', fmt=".2f", cbar=True)
plt.title('Spearman相关系数矩阵')
plt.show()

# 导出相关系数矩阵的表格
spearman_corr.to_csv('所有单品Spearman相关系数矩阵.csv', index=True, encoding='utf-8')

# 读取包含Spearman相关系数的CSV文件
spearman_corr = pd.read_csv('所有单品Spearman相关系数矩阵.csv', encoding='gbk', index_col=0)

# 创建一个与相关系数矩阵相同形状的DataFrame，填充为空白
filtered_corr = pd.DataFrame('', index=spearman_corr.index, columns=spearman_corr.columns)

# 使用循环遍历相关系数矩阵的每个单元格，筛选出符合条件的单元格
for i in range(len(spearman_corr.index)):
    for j in range(len(spearman_corr.columns)):
        corr_value = spearman_corr.iloc[i, j]
        if corr_value > 0.8 or corr_value < -0.7:
            filtered_corr.iloc[i, j] = corr_value

# 将筛选后的相关系数矩阵保存为CSV文件
filtered_corr.to_csv('强相关的相关系数矩阵.csv', encoding='utf-8')

# 读取包含Spearman相关系数的CSV文件
spearman_corr = pd.read_csv('强相关的相关系数矩阵.csv', encoding='utf-8', index_col=0)

# 初始化一个列表来存储符合条件的单品对及相关系数
strong_correlations = []

# 使用循环遍历相关系数矩阵的每个单元格，筛选出大于0.8或小于-0.8的单品对及相关系数
for i in range(len(spearman_corr.index)):
    for j in range(len(spearman_corr.columns)):
        corr_value = spearman_corr.iloc[i, j]

```

```

        if corr_value != '' and (corr_value > 0.8 or corr_value < -0.7):
            product1 = spearman_corr.index[i]
            product2 = spearman_corr.columns[j]
            strong_correlations.append([product1, product2, corr_value])

# 创建一个DataFrame来存储强相关的单品对及相关系数
strong_correlations_df = pd.DataFrame(strong_correlations, columns=['单品1', '单品2',
    '相关系数'])

# 将强相关的单品对及相关系数保存为CSV文件
strong_correlations_df.to_csv('强相关的单品对及相关系数.csv', index=False, encoding='utf-8')

# 读取包含单品对及相关系数的CSV文件
strong_correlations_df = pd.read_csv('强相关的单品对及相关系数.csv', encoding='utf-8')

# 对单品对按照字母顺序排序
strong_correlations_df[['单品1', '单品2']] = np.sort(strong_correlations_df[['单品1',
    '单品2']], axis=1)

# 删除重复的单品对，保留第一个出现的
strong_correlations_df = strong_correlations_df.drop_duplicates(subset=['单品1', '单品2'],
    keep='first')

# 保存删除重复单品对后的数据为新的CSV文件
strong_correlations_df.to_csv('强相关的单品对.csv', index=False, encoding='utf-8')

# 读取包含强相关的单品对的CSV文件
strong_correlations_df = pd.read_csv('强相关的单品对.csv', encoding='gbk')

# 读取包含分类名称的附件1表格（假设该表格的文件名为附件1.xlsx）
categories_df = pd.read_excel('附件1.xlsx')

# 将强相关的单品对表格与分类名称表格进行连接，使用单品1和单品2列进行连接
result_df = strong_correlations_df.merge(categories_df, left_on='单品1', right_on='单品名称',
    how='left')
result_df.rename(columns={'分类名称': '单品1分类名称'}, inplace=True)
result_df = result_df.merge(categories_df, left_on='单品2', right_on='单品名称', how='left')
result_df.rename(columns={'分类名称': '单品2分类名称'}, inplace=True)

# 保存连接后的结果为新的CSV文件
result_df.to_csv('互补替代单品对.csv', index=False, encoding='utf-8')

```

Listing 2. 问题二

```
import random
```

```

import numpy as np
from deap import base, creator, tools, algorithms
import matplotlib.pyplot as plt

# 创建适应度函数的最大化目标
creator.create("FitnessMax", base.Fitness, weights=(1.0,))

# 创建个体（染色体）类
creator.create("Individual", list, fitness=creator.FitnessMax)

# 定义问题参数
num_genes = 6 # 决策变量的数量

# 定义目标函数（适应度函数）
def objective_function(individual):
    # 对应决策变量
    x = individual

    c1 = 0.004 # 常数c1
    c2 = [12.3533066, 3.225434281, 7.925177134, 4.58402071, 3.614925942, 3.724212133] # c2数组
    a = [25.452, 279.4170, 69.2915, 9.3025, 59.0226, 106.723] # a
    b = [0.0421, -0.0778, -0.0639, 0.0898, 0.0395, -0.0445] # b
    loss = [0.11975, 0.1028, 0.14142, 0.07122, 0.08515, 0.08131] # loss数组
    alpha = [0.276178142, 0.614610374, 0.499660436, 0.681622204, 0.715190645, 0.46527093] #
        alpha数组

    total_value = 0
    for i in range(num_genes):
        total_value += (((c1 / x[i] + c2[i]) * (1 + alpha[i]) - c2[i]) * a[i] * np.exp(b[i] *
            ((c1 / x[i] + c2[i]) * (1 + alpha[i])))) - loss[i] * x[i] *
            c2[i])

    return total_value,

# 定义约束条件函数
def constraint_1(individual):
    # 对应决策变量
    x = individual

    qmax = [18.90426096, 146.0112251, 26.23555554, 24.50013194, 108.932873, 53.35858281]
    qmin = [15.4671226, 119.4637296, 21.46545453, 20.0455625, 89.12689608, 43.6570223]

    for i in range(num_genes):
        if x[i] < qmin[i] or x[i] > qmax[i]:
            return False

```

```

return True

# 定义遗传算法的进化过程
def genetic_algorithm(pop_size, num_generations):
    toolbox = base.Toolbox()

    # 注册每个决策变量的范围
    qmax = [18.90426096, 146.0112251, 26.23555554, 24.50013194, 108.932873, 53.35858281]
    qmin = [15.4671226, 119.4637296, 21.46545453, 20.0455625, 89.12689608, 43.6570223]

    for i in range(num_genes):
        gene_name = f"gene{i + 1}"
        toolbox.register(gene_name, random.uniform, qmin[i], qmax[i])

    toolbox.register("individual", tools.initCycle, creator.Individual, (toolbox.gene1,
        toolbox.gene2, toolbox.gene3, toolbox.gene4, toolbox.gene5, toolbox.gene6), n=1)

    toolbox.register("population", tools.initRepeat, list, toolbox.individual)
    toolbox.register("evaluate", objective_function)
    toolbox.decorate("evaluate", tools.DeltaPenalty(constraint_1, (0.0,)))
    toolbox.register("mate", tools.cxSimulatedBinaryBounded, low=[1] * num_genes, up=[10] *
        num_genes, eta=1.0)
    toolbox.register("mutate", tools.mutPolynomialBounded, low=[1] * num_genes, up=[10] *
        num_genes, eta=1.0, indpb=0.2)
    toolbox.register("select", tools.selTournament, tournsize=3)

    population = toolbox.population(n=pop_size)

    algorithms.eaSimple(population, toolbox, cxpb=0.7, mutpb=0.2, ngen=num_generations,
        verbose=False)
    # 初始化迭代收敛曲线数据
    best_fitness_values = []

    for gen in range(num_generations):
        # 迭代进化
        algorithms.eaSimple(population, toolbox, cxpb=0.7, mutpb=0.2, ngen=1, verbose=False)

        # 记录每一代的最佳适应度值
        best_individual = tools.selBest(population, k=1)[0]
        best_fitness = best_individual.fitness.values[0]
        best_fitness_values.append(best_fitness)

        print(f"代数 {gen + 1}/{num_generations} - 最佳适应度值: {best_fitness}")

    # 绘制迭代收敛曲线
    plt.figure(figsize=(10, 6))

```



```

plt.plot(range(1, num_generations + 1), best_fitness_values, marker='o', linestyle='-')
plt.title("迭代收敛曲线")
plt.xlabel("代数")
plt.ylabel("最佳适应度值")
plt.grid(True)
plt.show()

best_individual = tools.selBest(population, k=1)[0]
best_fitness = best_individual.fitness.values[0]

return best_individual, best_fitness

if __name__ == "__main__":
    # 设置随机数生成器的种子, 例如, 使用种子值11
    random.seed(11)

    pop_size = 50 # 种群大小
    num_generations = 500 # 进化的代数

    best_solution, best_fitness = genetic_algorithm(pop_size, num_generations)

    print("最佳解决方案:", best_solution)
    print("最佳适应度值:", best_fitness)

from scipy.optimize import curve_fit
from sklearn.metrics import r2_score
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA

# 读取包含销售数据的 CSV 文件
df = pd.read_csv("异常值处理3.csv", encoding='utf-8')

# 将销售日期列转换为日期类型
df['销售日期'] = pd.to_datetime(df['销售日期'])

# 获取唯一的分类名称
unique_categories = df['分类名称'].unique()

# 创建一个结果 DataFrame, 包含销售日期和各个分类的列
result_df = pd.DataFrame(columns=['销售日期'] + list(unique_categories))

# 循环遍历每个日期, 计算每天的类别加权售价

```

```

dates = df['销售日期'].unique()
for date in dates:
    # 获取当天的数据
    date_data = df[df['销售日期'] == date]

    # 计算当天的总销量
    total_sales = date_data['销量(千克)'].sum()

    # 初始化一个字典，用于存储各个类别的加权售价
    weighted_prices = {}

    # 遍历每个类别，计算加权售价
    for category in unique_categories:
        category_data = date_data[date_data['分类名称'] == category]
        # 计算当天的总销量
        total_sales = category_data['销量(千克)'].sum()
        category_sales = category_data['销量(千克)'].sum()
        category_weighted_price = (
            category_data['销售单价(元/千克)'] * (category_data['销量(千克)'] /
            total_sales)).sum()
        weighted_prices[category] = category_weighted_price

    # 创建一个 DataFrame，用于存储当天的结果
    daily_result = {'销售日期': date, **weighted_prices}

    # 将当天的结果添加到结果 DataFrame 中
    result_df = pd.concat([result_df, pd.DataFrame([daily_result])], ignore_index=True)

# 将结果保存到 CSV 文件
result_df.to_csv("加权平均售价.csv", index=False, encoding='utf-8')

# 修改matplotlib字体设置，支持中文显示 (Windows)
plt.rcParams['font.sans-serif'] = ['SimHei'] # 或者 ['Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 读取CSV文件
data = pd.read_csv('7月.csv', encoding='gbk')

# 提取销量和售价列数据
sales = data['销量'].values
price = data['售价'].values

# 定义指数函数模型
def exponential_func(x, a, b):
    return a * np.exp(b / x)

```

```

# 使用curve_fit拟合数据
params, covariance = curve_fit(exponential_func, price, sales)

# 提取拟合的参数
a, b = params

# 生成平滑的拟合曲线
x_smooth = np.linspace(min(price), max(price), 100)
y_smooth = exponential_func(x_smooth, a, b)

# 提取拟合的参数
a, b = params

# 计算R-squared值
y_predicted = exponential_func(price, a, b)
r_squared = r2_score(sales, y_predicted)
print(a)
print(b)
# 输出R-squared值
print(f'R-squared值: {r_squared:.2f}')
# 设置字体系列为Times New Roman
annot_kws = {'family': 'Times New Roman', 'size': 12, 'weight': 'normal'}

# 绘制实际数据的散点图和拟合曲线
plt.scatter(price, sales, label='实际数据')
plt.plot(x_smooth, y_smooth, color='red', linewidth=2, label='拟合曲线')

plt.title('水生根茎类销量随售价变化拟合曲线', fontsize=16)
# 添加标签和图例
plt.xlabel('售价', fontsize=12)
plt.ylabel('销量', fontsize=12)
plt.legend()

# 显示图形
plt.show()

# 读取附件4.xlsx和附件1.xlsx
data4 = pd.read_csv('附件4.csv', encoding='gbk')
data1 = pd.read_csv('附件1.csv', encoding='gbk')

# 使用merge函数将附件4和附件1连接在一起，基于单品名称列进行连接
joined_data = data4.merge(data1, on='单品名称')

# 导出连接后的数据为CSV文件
joined_data.to_csv('损耗率.csv', index=False, encoding='utf-8-sig')

```

```

# 修改matplotlib字体设置, 支持中文显示 (Windows)
plt.rcParams['font.sans-serif'] = ['SimHei'] # 或者 ['Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题


# 尝试不同的编码方式读取CSV文件
possible_encodings = ['utf-8', 'utf-16', 'gbk', 'latin-1', 'utf-8-sig']
for encoding in possible_encodings:
    try:
        data = pd.read_csv("Arima.csv", encoding=encoding)
        break
    except UnicodeDecodeError:
        continue
else:
    raise ValueError("无法使用已知编码方式解码CSV文件。")


# 将日期列转换为日期格式
data['销售日期'] = pd.to_datetime(data['销售日期'])

# 设置日期列为索引
data.set_index('销售日期', inplace=True)

# 创建时间序列变量
time_series = data['水生根茎类3']

# 定义平稳性检验函数

def check_stationarity(timeseries):

    # 进行Dickey-Fuller检验
    result = adfuller(timeseries, autolag='AIC')
    print("ADF Test Results:")
    print(f'ADF Statistic: {result[0]}')
    print(f'p-value: {result[1]}')
    print(f'Critical Values:')
    for key, value in result[4].items():
        print(f' {key}: {value}')

```

```

if result[1] <= 0.05:
    print("数据平稳，无需差分处理。")
    d = 0
    return timeseries, d
else:
    print("数据非平稳，进行一阶差分处理。")
    d = 1
    first_diff = timeseries.diff().dropna()

    # 进行一阶差分后再次进行平稳性检验
    result_diff = adfuller(first_diff, autolag='AIC')
    print("一阶差分后的平稳性检验结果:")
    print(f'ADF Statistic: {result_diff[0]}')
    print(f'p-value: {result_diff[1]}')
    print(f'Critical Values:')
    for key, value in result_diff[4].items():
        print(f' {key}: {value}')

    if result_diff[1] <= 0.05:
        print("一阶差分后数据平稳，无需二阶差分处理。")
        return first_diff, d
    else:
        print("一阶差分后数据仍非平稳，进行二阶差分处理。")
        d = 2
        second_diff = first_diff.diff().dropna()

        # 进行二阶差分后再次进行平稳性检验
        result_diff = adfuller(second_diff, autolag='AIC')
        print("二阶差分后的平稳性检验结果:")
        print(f'ADF Statistic: {result_diff[0]}')
        print(f'p-value: {result_diff[1]}')
        print(f'Critical Values:')
        for key, value in result_diff[4].items():
            print(f' {key}: {value}')

        if result_diff[1] <= 0.05:
            print("二阶差分后数据平稳。")
            return second_diff, d
        else:
            print("经过二阶差分后数据仍非平稳，建议进一步处理。")
            return None, None

# 进行平稳性检验
stationary_time_series, d = check_stationarity(time_series)

def white_noise_test(data, lags_list):

```

```

from statsmodels.stats.diagnostic import acorr_ljungbox

for lag in lags_list:
    re = acorr_ljungbox(data, lags=lag)
    print(f"延迟阶数: {lag}")
    print("卡方统计量:", re.lb_stat)
    print("P值:", re.lb_pvalue)

    for p in re.lb_pvalue:
        if p <= 0.05:
            print("序列不是白噪声。")
            return True

    print("序列为白噪声。")
    return False
lags_list=[9]
result=white_noise_test(time_series,lags_list)

if stationary_time_series is not None :
    # 绘制差分后的ACF和PACF图
    plt.figure(figsize=(12, 6))
    plot_acf(stationary_time_series, lags=20, title='ACF 自相关函数图')
    plt.show()

    plt.figure(figsize=(12, 6))
    plot_pacf(stationary_time_series, lags=20, title='PACF 偏自相关函数图')
    plt.show()

    # 网格搜索选择ARIMA模型的阶数
    best_aic = np.inf
    best_order = None

    for p in range(3):
        for q in range(10):
            try:
                model = ARIMA(stationary_time_series, order=(p, d, q))
                results = model.fit()
                aic = results.aic

                if aic < best_aic:
                    best_aic = aic
                    best_order = (p, d, q)

            except:
                continue

# 输出选择的最优阶数

```

```

print(f"选择的最优阶数为: {best_order}, AIC值为: {best_aic}")

# 构建最优ARIMA模型
model = ARIMA(stationary_time_series, order=best_order)
results = model.fit()

print(stationary_time_series)

model_summary = results.summary()

# # 获取自回归参数 (AR)
# ar_params = results.arparams
# # 将AR参数转换为列表并打印
# print("自回归参数 (AR):", list(ar_params))
# # 获取滑动平均参数 (MA)
# ma_params = results.maparams
# # 将MA参数转换为列表, 并使用join()方法将其连接成一个字符串, 然后打印
# ma_params_str = ", ".join(map(str, ma_params))
# print("滑动平均参数 (MA): " + ma_params_str)
#
# # 获取残差 (Residuals)
# residuals = results.resid
# # 将残差转换为列表并打印
# residuals_list = list(residuals)
# print("残差:", residuals_list)
#
# # 计算残差的方差
# residual_variance = np.var(residuals)
# print("残差方差:", residual_variance)

# 将模型参数转换为DataFrame
model_summary_df = pd.DataFrame(model_summary.tables[1].data[1:],
                                columns=model_summary.tables[1].data[0])

# 保存模型参数为CSV文件
model_summary_file_path = "model_summary.csv"
model_summary_df.to_csv(model_summary_file_path, index=False)

# 绘制拟合图形
plt.figure(figsize=(10, 6))
plt.plot(stationary_time_series, label='Actual')
plt.plot(results.fittedvalues, color='red', label='Fitted')
plt.legend()
plt.title('ARIMA 拟合预测加成率')
plt.show()

# 预测未来时间步数的结果

```

```

forecast_steps = 7
forecast_values = results.forecast(steps=forecast_steps)

# 如果进行了差分，将预测结果逆差分得到原始数据的尺度
if d > 0:
    forecast_values_original_scale = time_series.iloc[-1] + forecast_values.cumsum()
else:
    forecast_values_original_scale = forecast_values

# 获取模型对历史数据的拟合值
fitted_values = results.fittedvalues
# 获取残差
residuals = stationary_time_series - fitted_values

# 检查白噪声
lags_list = [9]
result = white_noise_test(residuals, lags_list)

# 计算R方（拟合优度）
def calculate_r_squared(actual, predicted):
    # 计算总平均值
    mean_actual = np.mean(actual)
    # 计算总平方和
    total_sum_of_squares = np.sum((actual - mean_actual) ** 2)
    # 计算残差平方和
    residual_sum_of_squares = np.sum((actual - predicted) ** 2)
    # 计算R方
    r_squared = 1 - (residual_sum_of_squares / total_sum_of_squares)
    return r_squared

# 计算R方
r_squared = calculate_r_squared(stationary_time_series, fitted_values)
print(f"拟合优度 (R方) : {r_squared:.2f}")

```

Listing 3. 问题三

```

import pandas as pd
import random
import numpy as np
from deap import base, creator, tools, algorithms
import matplotlib.pyplot as plt

```



```

# 读取进价表格.csv文件
df = pd.read_csv('6.24-6.30单品销售量.csv',encoding='gbk')

# 获取单品名称列（除去第一行）
item_names = df.columns[1:]

# 定义GM(1,1)灰色预测模型
def GM_11(x0):
    n = len(x0)
    x1 = np.cumsum(x0)
    Z1 = np.zeros((n-1, 2))
    for i in range(1, n):
        Z1[i-1][0] = -0.5 * (x1[i] + x1[i-1])
        Z1[i-1][1] = 1
    B = np.linalg.inv(Z1.T @ Z1) @ Z1.T @ x0[1:]
    a, u = B[0], B[1]
    X_ = np.zeros(n)
    X_[0] = x0[0]
    for i in range(1, n):
        X_[i] = (x0[0] - u/a) * (1 - np.exp(a)) * np.exp(-a*(i))
    return X_

# 预测未来1天的值
predicted_values = {}
for item_name in item_names:
    x0 = df[item_name].values
    predicted_value = GM_11(x0)[-1]
    predicted_values[item_name] = predicted_value

# 创建包含预测结果的DataFrame
prediction_df = pd.DataFrame.from_dict(predicted_values, orient='index', columns=['预测值'])

# 导出结果为CSV文件
prediction_df.to_csv('销售量灰色预测.csv', index=True)

plt.rcParams['font.sans-serif'] = ['SimHei'] # 或者 ['Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 创建适应度函数的最大化目标
creator.create("FitnessMax", base.Fitness, weights=(1.0,))

# 创建个体（染色体）类
creator.create("Individual", list, fitness=creator.FitnessMax)

# 定义问题参数
num_genes = 49 # 决策变量的数量

```

```

# 定义目标函数 (适应度函数)
def objective_function(individual):
    # 对应决策变量
    x = individual

    c1 = 0.004 # 常数c1
    c2 = [
        2.11, 7.59, 15.6, 12.13, 1.3, 7, 8.39, 3.43, 8.97, 2.84,
        3.49, 3.63, 4.09, 10.38, 2.33, 2.56, 2.15, 4.29, 9.16, 3.41,
        4.07, 2.6, 3.12, 4.4, 2.7, 2.1, 18, 3.32, 6.68, 2.21, 11.67,
        1.45, 1.95, 3.29, 4.62, 12.72, 3.12, 2.6, 1.53, 3.2, 2.55, 3.98,
        2.66, 4.34, 5.29, 9.66, 9.9, 13.69, 16.06
    ] # c2数组
    a = 106.723 # a
    b = -0.0445 # b
    loss = [
        0.0943, 0.0926, 0.108, 0.0943, 0.0943, 0.069, 0.0943, 0.0607, 0.1018, 0.0943,
        0.0943, 0.057, 0.1443, 0.0554, 0.0943, 0.1568, 0.1362, 0.0943, 0.0961, 0.002,
        0.0943, 0.0842, 0.0761, 0.0248, 0.1033, 0.0943, 0.2405, 0.0943, 0.1525, 0.1852,
        0.2925, 0.0045, 0, 0.0657, 0.137, 0.0943, 0.0084, 0.0943, 0.0943, 0.0671, 0.0943,
        0.0501, 0.1281, 0.0943, 0.1663, 0.1851, 0.2616, 0.0943, 0.1269
    ] # loss数组
    alpha = [
        1.720412579, 0.63297572, 0.538461538, 0.598011048, 3.846153846, 0.716734694,
        0.466848118, 0.656809663, 0.436215958, 0.450001818, 0.239266345, 0.500196773,
        0.948305973, 0.347481071, 0.959600762, 0.887369792, 0.662458472, 0.393789669,
        0.526928675, 0.470081951, 0.307197379, 0.833516484, 0.729190505, 0.406606223,
        0.877777778, 0.507518457, 0.14934498, 0.726549053, 0.518712575, 0.641887524,
        0.170697596, 0.280581169, 0.390316497, 1.132218845, 0.301406926, 0.46079655,
        0.65357906, 0.440769231, 1.549019608, 0.7984375, 0.576470588, 0.490368509,
        1.250626566, 1.073732719, 0.680529301, 0.451345756, 0.424915825, 0.239277656,
        0.618721461
    ] # alpha数组

    total_value = 0
    for i in range(num_genes):
        total_value += 0.1*(((c1 / x[i] + c2[i]) * (1 + alpha[i]) - c2[i]) * a * np.exp(b *
            ((c1 / x[i] + c2[i]) * (1 + alpha[i]))) - loss[i] * x[i] * \
            c2[i]))

    return total_value,

# 定义约束条件函数
def constraint_1(individual):
    # 对应决策变量
    x = individual

```

```

qmax = [
    25.3, 18.59, 7.2292, 7.7396, 10.67, 9.6679, 12.3057, 15.8015, 9.317, 27.5,
    39.6, 17.4856, 7.7396, 7.084, 9.9, 9.9484, 14.1724, 11, 3.7356, 9.9,
    12.1, 6.468, 5.7035, 6.6, 5.1799, 11, 4.7729, 4.4, 3.1944, 7.3986,
    4.158, 14.3, 6.6, 1.1, 2.9711, 2.112, 2.2, 2.2, 1.1, 1.5136, 1.1,
    1.034, 0.3223, 0.5632, 0.7931, 1.4025, 1.0956, 0.5632, 0.2541
]

qmin = [
    20.7, 15.21, 5.9148, 6.3324, 8.73, 7.9101, 10.0683, 12.9285, 7.623, 22.5,
    32.4, 14.3064, 6.3324, 5.796, 8.1, 8.1396, 11.5956, 9, 3.0564, 8.1,
    9.9, 5.292, 4.6665, 5.4, 4.2381, 9, 3.9051, 3.6, 2.6136, 6.0534,
    3.402, 11.7, 5.4, 0.9, 2.4309, 1.728, 1.8, 1.8, 0.9, 1.2384, 0.9,
    0.846, 0.2637, 0.4608, 0.6489, 1.1475, 0.8964, 0.4608, 0.2079
]

for i in range(num_genes):
    if x[i] < qmin[i] or x[i] > qmax[i]:
        return False
return True

# 定义遗传算法的进化过程
def genetic_algorithm(pop_size, num_generations):
    toolbox = base.Toolbox()

    qmax = [
        25.3, 18.59, 7.2292, 7.7396, 10.67, 9.6679, 12.3057, 15.8015, 9.317, 27.5,
        39.6, 17.4856, 7.7396, 7.084, 9.9, 9.9484, 14.1724, 11, 3.7356, 9.9,
        12.1, 6.468, 5.7035, 6.6, 5.1799, 11, 4.7729, 4.4, 3.1944, 7.3986,
        4.158, 14.3, 6.6, 1.1, 2.9711, 2.112, 2.2, 2.2, 1.1, 1.5136, 1.1,
        1.034, 0.3223, 0.5632, 0.7931, 1.4025, 1.0956, 0.5632, 0.2541
    ]

    qmin = [
        20.7, 15.21, 5.9148, 6.3324, 8.73, 7.9101, 10.0683, 12.9285, 7.623, 22.5,
        32.4, 14.3064, 6.3324, 5.796, 8.1, 8.1396, 11.5956, 9, 3.0564, 8.1,
        9.9, 5.292, 4.6665, 5.4, 4.2381, 9, 3.9051, 3.6, 2.6136, 6.0534,
        3.402, 11.7, 5.4, 0.9, 2.4309, 1.728, 1.8, 1.8, 0.9, 1.2384, 0.9,
        0.846, 0.2637, 0.4608, 0.6489, 1.1475, 0.8964, 0.4608, 0.2079
    ]

    for i in range(num_genes):
        gene_name = f"gene{i + 1}"
        toolbox.register(gene_name, random.uniform, qmin[i], qmax[i])

    toolbox.register("individual", tools.initCycle, creator.Individual,
        (toolbox.gene1, toolbox.gene2, toolbox.gene3, toolbox.gene4, toolbox.gene5,
        toolbox.gene6, toolbox.gene7, toolbox.gene8, toolbox.gene9, toolbox.gene10,

```

```

        toolbox.gene11, toolbox.gene12, toolbox.gene13, toolbox.gene14,
        toolbox.gene15,
        toolbox.gene16, toolbox.gene17, toolbox.gene18, toolbox.gene19,
        toolbox.gene20,
        toolbox.gene21, toolbox.gene22, toolbox.gene23, toolbox.gene24,
        toolbox.gene25,
        toolbox.gene26, toolbox.gene27, toolbox.gene28, toolbox.gene29,
        toolbox.gene30,
        toolbox.gene31, toolbox.gene32, toolbox.gene33, toolbox.gene34,
        toolbox.gene35,
        toolbox.gene36, toolbox.gene37, toolbox.gene38, toolbox.gene39,
        toolbox.gene40,
        toolbox.gene41, toolbox.gene42, toolbox.gene43, toolbox.gene44,
        toolbox.gene45,
        toolbox.gene46, toolbox.gene47, toolbox.gene48, toolbox.gene49), n=1)

toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("evaluate", objective_function)
toolbox.decorate("evaluate", tools.DeltaPenalty(constraint_1, (0.0,)))
toolbox.register("mate", tools.cxSimulatedBinaryBounded, low=[1] * num_genes, up=[10] *
    num_genes, eta=1.0)
toolbox.register("mutate", tools.mutPolynomialBounded, low=[1] * num_genes, up=[10] *
    num_genes, eta=1.0, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

population = toolbox.population(n=pop_size)

algorithms.eaSimple(population, toolbox, cxpb=0.7, mutpb=0.2, ngen=num_generations,
    verbose=False)

# 初始化迭代收敛曲线数据
best_fitness_values = []

for gen in range(num_generations):
    # 迭代进化
    algorithms.eaSimple(population, toolbox, cxpb=0.7, mutpb=0.2, ngen=1, verbose=False)

    # 记录每一代的最佳适应度值
    best_individual = tools.selBest(population, k=1)[0]
    best_fitness = best_individual.fitness.values[0]-203.93
    best_fitness_values.append(best_fitness)

    print(f"代数 {gen + 1}/{num_generations} - 最佳适应度值: {best_fitness}")

# 绘制迭代收敛曲线
plt.figure(figsize=(10, 6))

```

```

plt.plot(range(1, num_generations + 1), best_fitness_values, marker='o', linestyle='-')
plt.title("迭代收敛曲线",fontsize=16)
plt.xlabel("代数")
plt.ylabel("最佳适应度值")
plt.grid(True)
plt.show()

best_individual = tools.selBest(population, k=1)[0]
best_fitness = best_individual.fitness.values[0]

return best_individual, best_fitness

if __name__ == "__main__":
    # 设置随机数生成器的种子, 例如, 使用种子值11
    random.seed(11)

    pop_size = 50 # 种群大小
    num_generations = 500 # 进化的代数

    best_solution, best_fitness = genetic_algorithm(pop_size, num_generations)

    print("最佳解决方案:", best_solution)

```